



Combining Quantum with Physics-Inspired AI Models for Accelerating CFD Simulations

**A Seshaditya
Quasi**

non linear and lacks academic rigor



Outline

- **Introduction to Use Cases**

Silicon Crystal Growth Simulations

Cryogenic Vessels Design

- **Approaches**

Classical AI - PINN/PI-GANO/GAOT

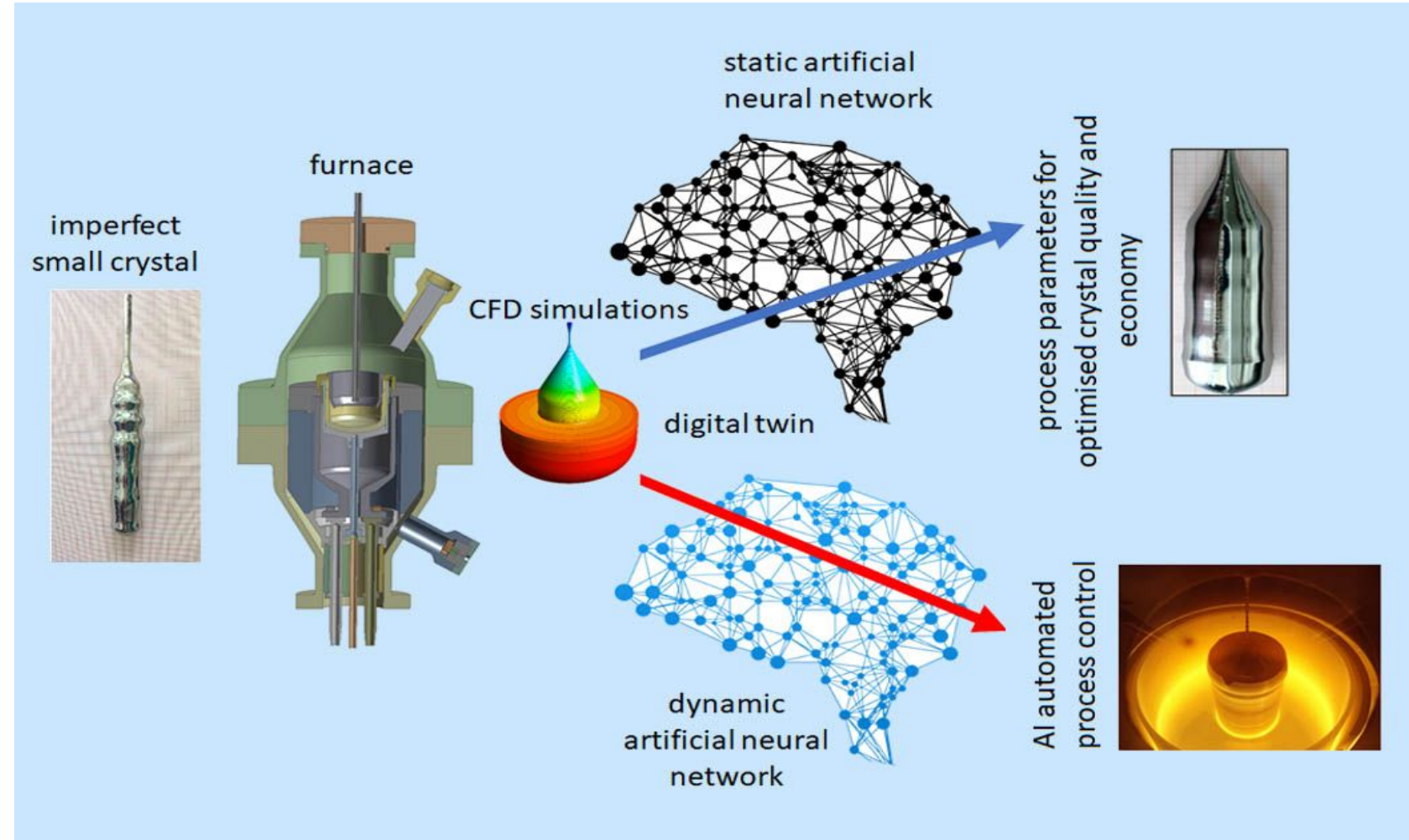
Hybrid-Quantum - QPINNs

- **Outlook**

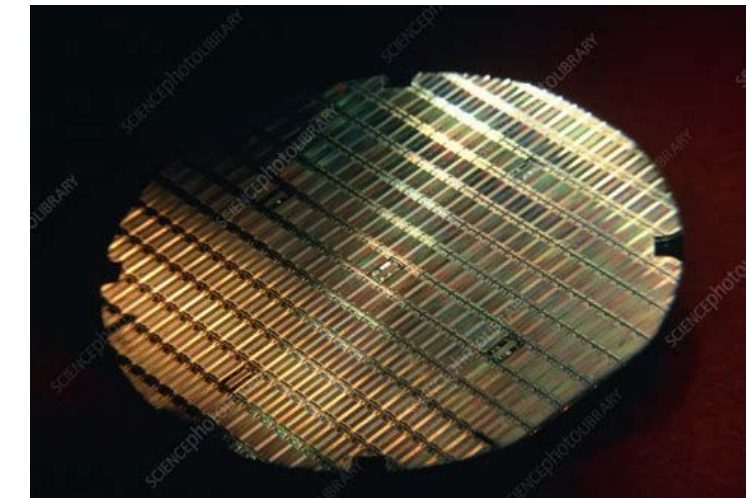
Silicon Crystal Growth

Accelerating CFD Simulations Real-Time Digital Twins

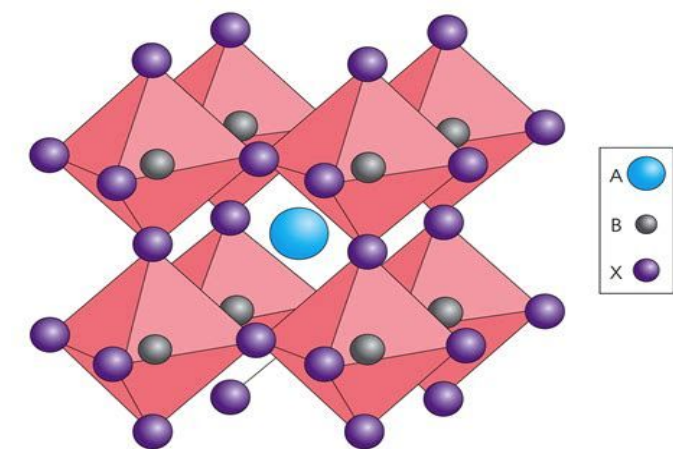
SDG 9
Ind, Inno
infra



SDG 12
Cons &
Prod



Silicon Wafers

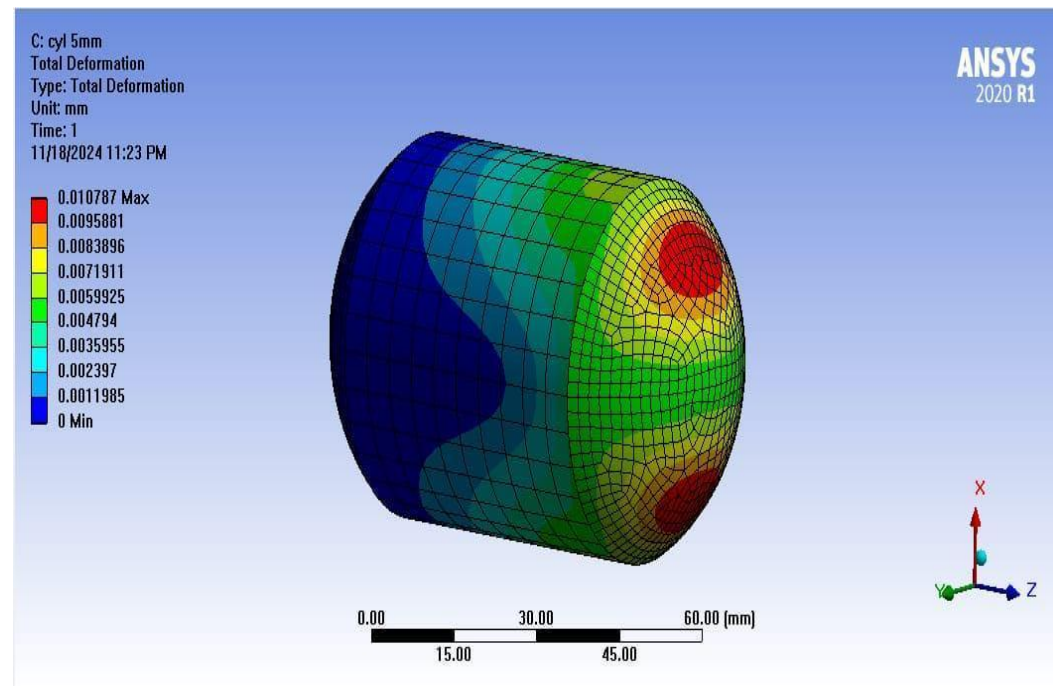


Perovskites

**Bringing Quantum Advantage through AI
(Hybrid Approaches on QPU/GPUs Hardware)**

Designing Cryogenic Vessels

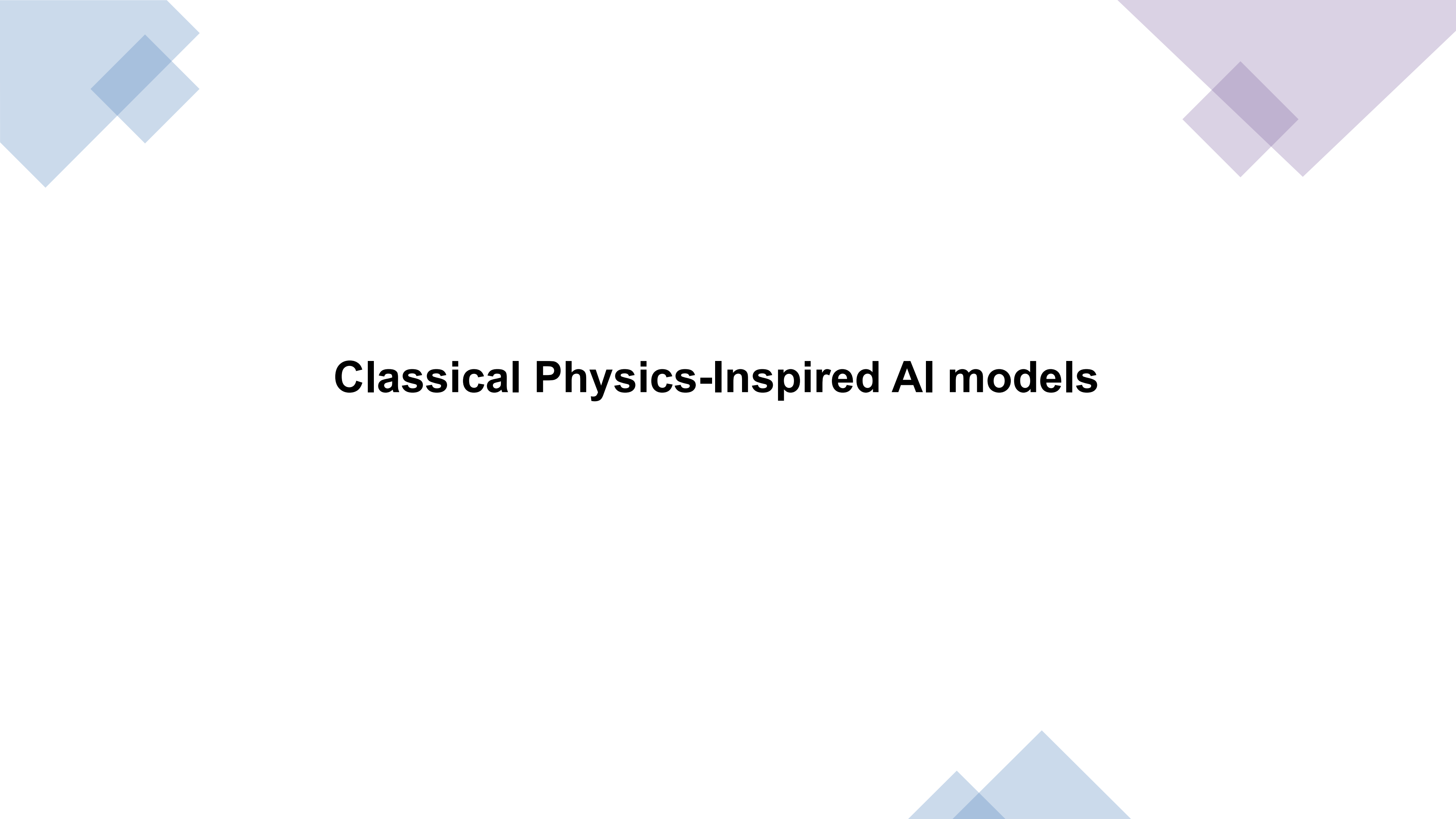
Accelerating CFD Reduces Design to Production Time



**Green Hydrogen
Economy**



Collaboration with Indo-Australian Startup

The slide features decorative geometric shapes in the corners. The top-left corner has two overlapping light blue squares. The top-right corner has two overlapping light purple squares. The bottom-right corner has two overlapping light blue squares. The text is centered in the middle of the slide.

Classical Physics-Inspired AI models

PINNs: Combining Physics

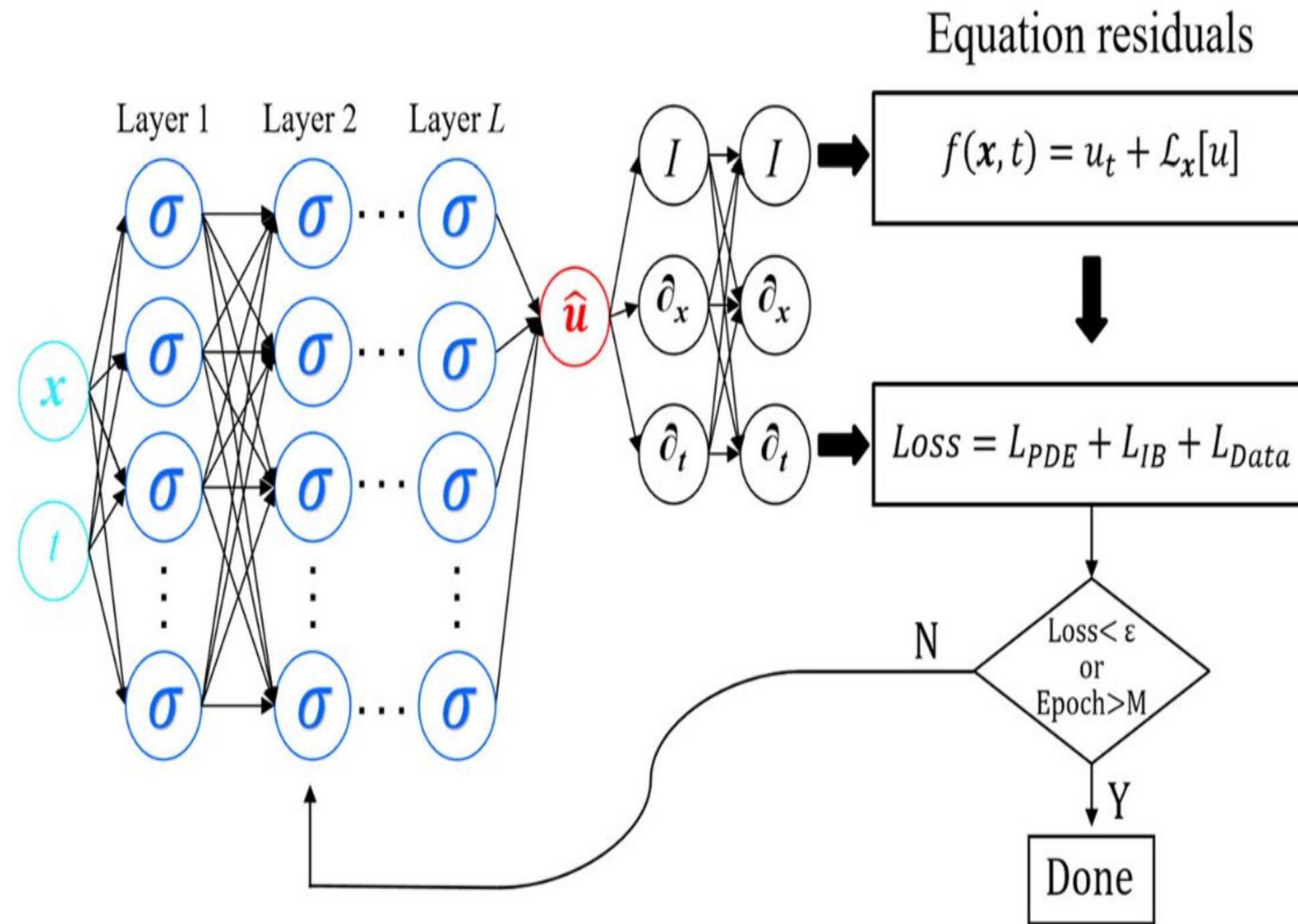


FIG. 1. Schematic diagram of PINN.

Multi-Physics Capability with Less Data or Conditions

PINNs: Combining Physics

PINNs

- Solve PDE of the form
 $\mathcal{L}(\mathbf{u}) = \mathbf{0}$ on domain Ω with BCs and ICs.
- Define trial solution $u_\theta(x)$.
- Minimize loss:

$$\mathcal{L} = \lambda_f \|\mathcal{L}(u_\theta)\|_{\Omega_f}^2 + \lambda_b \|u_\theta - g\|_{\partial\Omega}^2 + \lambda_i \|u_\theta - u_0\|_{t=0}^2$$

Essentially: They optimize a differentiable surrogate that satisfies the PDE.

Multi-Physics Capability with Less Data or Conditions

PINO: Physics Inspired Neural Operators

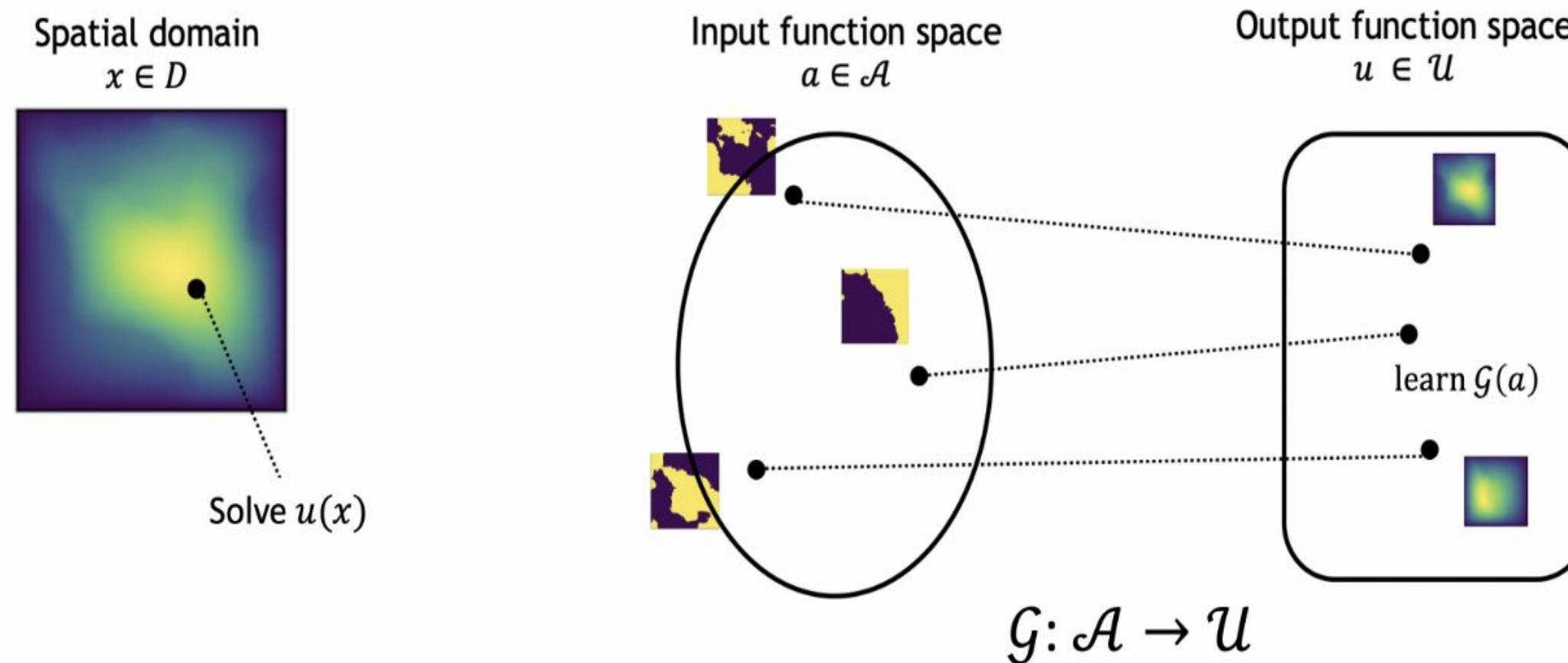


Figure 3: solve for one specific instance verse learn the entire solution operator
Left: numerical solvers and PINNs focus on solving one specific instance. Right: neural operators learn the solution operator for a family of equations.

Higher Accuracy & Multi-Physics Capability with Less Data

PINO: Physics Inspired Neural Operators

PINOs

- Solve families of PDEs:
 $\mathcal{L}(\mathbf{u}; \mathbf{a}) = \mathbf{0}$, parameterized by $\mathbf{a}$ (e.g., coefficients, BCs, ICs).
- Learn a neural operator

$$u = \mathcal{G}_\theta(a)$$

- Use integral kernels or spectral convolution layers (FNO, GNO).
- Physics-informed loss includes:

$$\mathcal{L} = \|\mathcal{L}(\mathcal{G}_\theta(a))\|^2 + \text{data terms (optional)}.$$

Key: The operator learns solution *distributions*, not single functions.

Higher Generalisability Data

Comparison

★ Why Use PINO Instead of PINNs?

PINO combines the advantages of both:

Feature	PINO	PINN
Learns mappings between <i>functions</i>	✓	✗
Good for fast prediction on new inputs	✓	✗
Uses physics constraints	✓	✓
Data-efficiency	High	Medium
Computational cost during inference	Very low	High

Comparison

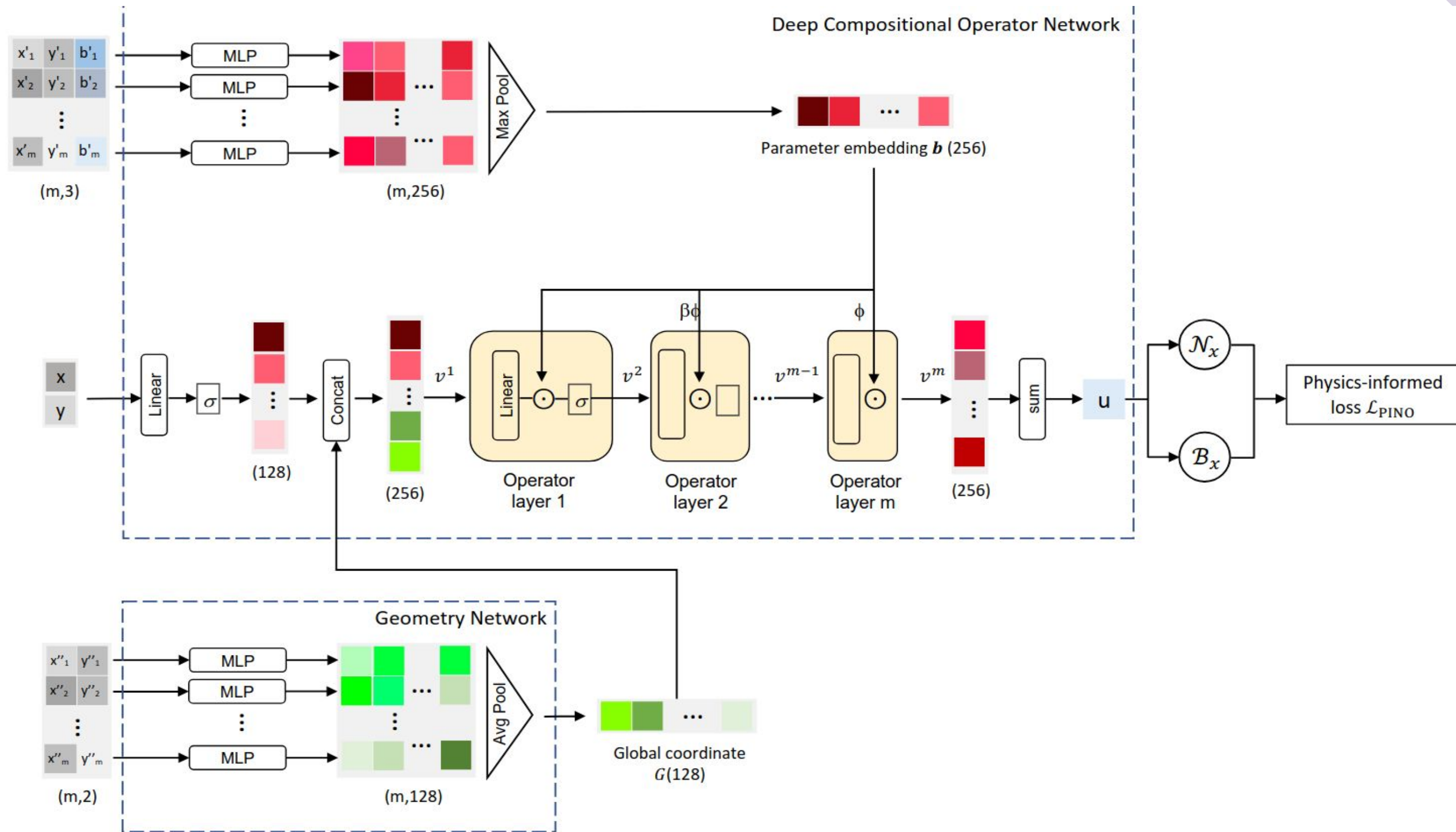
5. Generalization Ability

Category	PINNs	PINOs
Generalization to new BCs/ICs	Very limited unless retrained	Very good—operator learns the mapping
Generalization to new geometries	Poor	Better (with geometry-aware operators)
Resolution independence	No	Often yes (esp. Fourier neural operators)

PINOs are explicitly designed for **out-of-distribution generalization** across PDE input conditions.

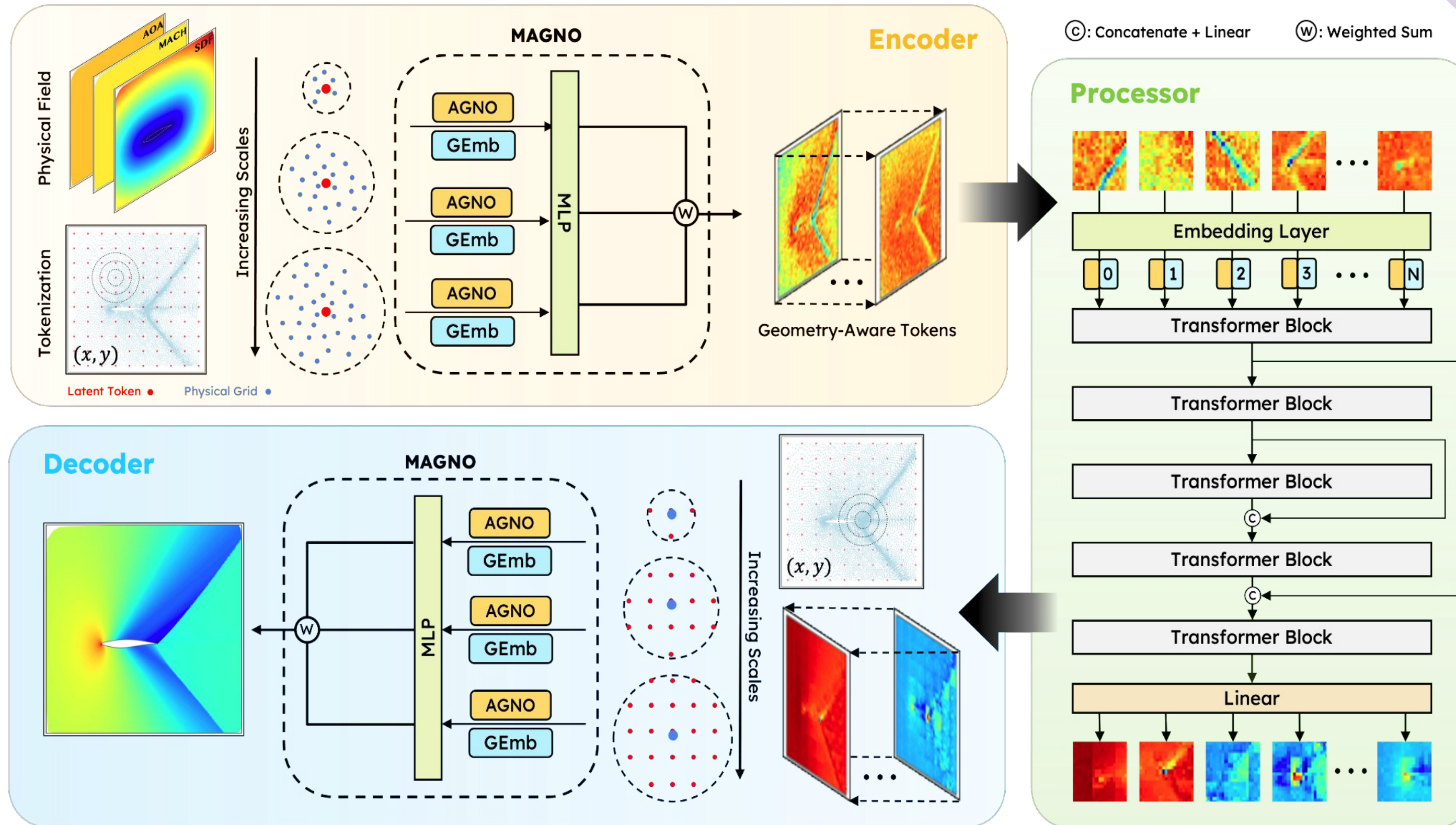


PI- GANO: Geometry Aware Neural Operator



Generalize across PDE parameter representations and geometries

GAOT: Geometry Aware Operator Transformers

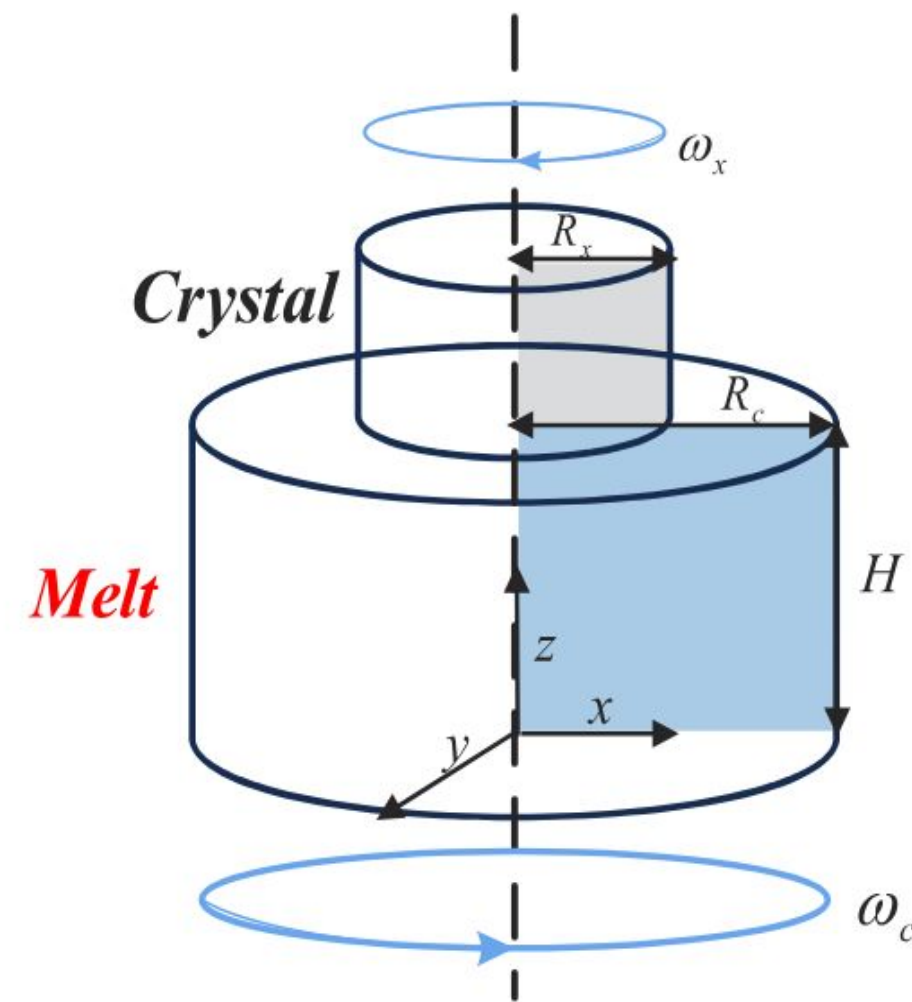


Multiscale Attentional Graph Neural Operator (MAGNO)

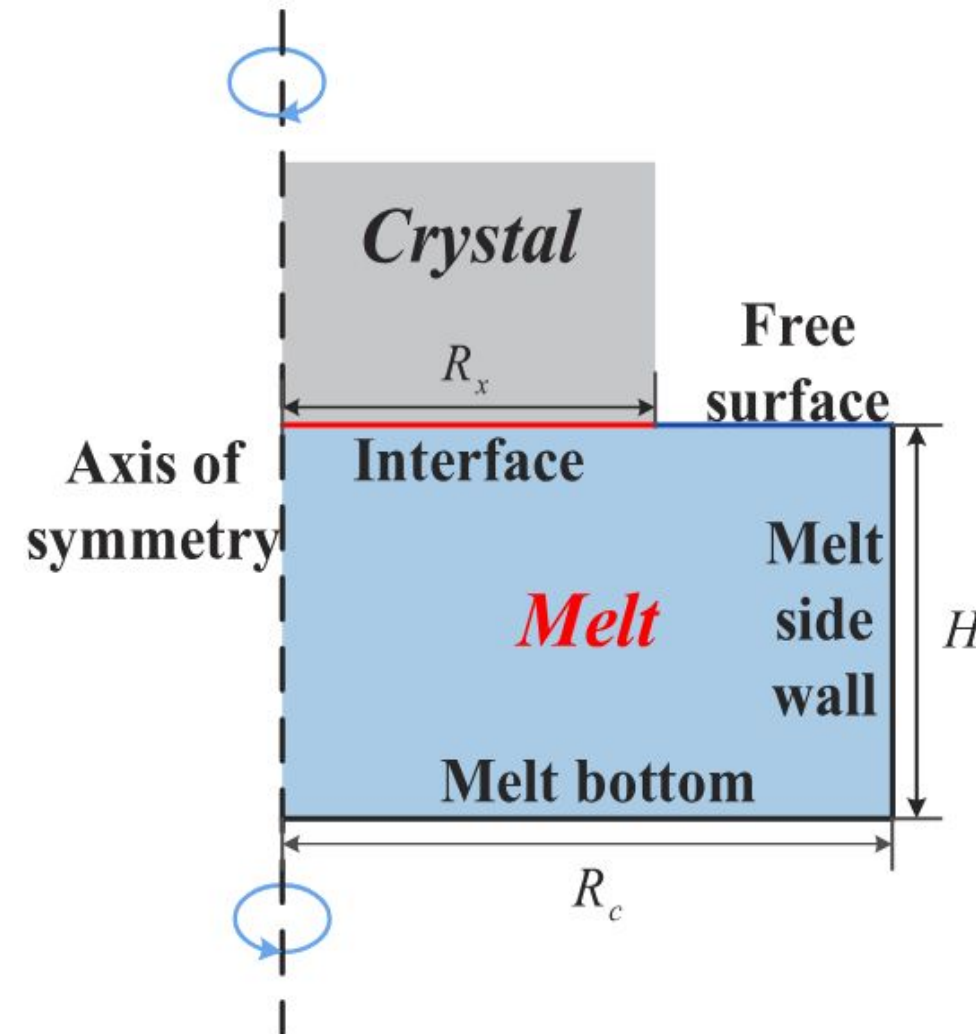
Comparison

Aspect	PI-GANO	GAOT (e.g., GINO)
Physics-Informed	Yes (can train without labeled data)	No (typically requires labeled simulation data)
Geometry Handling	Explicit geometry encoder, geometry-aware	SDF and point-cloud encoding, geometry-aware
Data Efficiency	High (physics-informed, less data needed)	Lower (relies on simulation data for training)
Generalization	Across geometry and PDE parameters	Across geometry (parameters less emphasized)
Computational Speed	Fast (neural operator-based)	Extremely fast (demonstrated 26,000x speed-up)

Silicon Crystal Growth (CZ process)



(a)



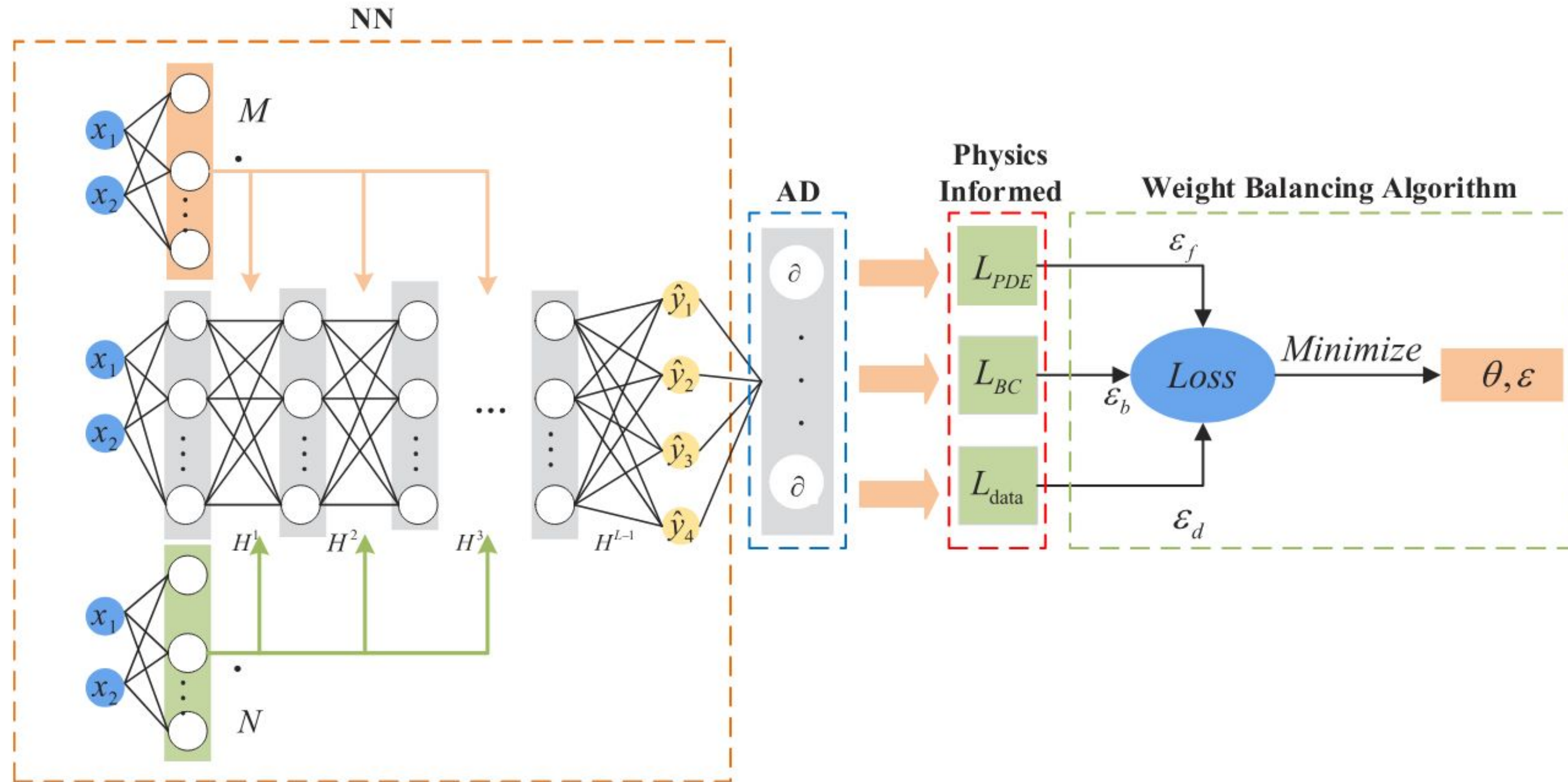
(b)

$$\frac{\partial v_r}{\partial r} + \frac{v_r}{r} + \frac{\partial v_z}{\partial z} = 0, \quad (38)$$

$$v_r \cdot \frac{\partial v_r}{\partial r} - \frac{v_\theta^2}{r} + v_z \cdot \frac{\partial v_r}{\partial z} = -\frac{\partial P}{\partial r} + \frac{1}{\text{Re}} \left(\frac{\partial^2 v_r}{\partial r^2} + \frac{\partial v_r}{r \partial r} - \frac{v_r}{r^2} + \frac{\partial^2 v_r}{\partial z^2} \right), \quad (39)$$

Navier–Stokes(NS), Particle Transport, and Heat Transfer equation

Spatial Information & Adaptive Loss Balance PINN



Optimizes the thermal-fluid coupling and Improves training stability

Spatial Information & Adaptive Loss Balance PINN

1. The introduction of spatial coordinate information

Building upon the original multi-layer perceptron (MLP) architecture, the PINN reintroduces the original spatial coordinate information into each hidden layer and incorporates an additional bias vector at each layer to enhance model expressiveness. The updated formulation for the hidden layer is given as

$$H^l = \sigma(W^l H^{l-1} + b^l) \bullet X + b_a, \quad (14)$$

where $\bullet$ represents multiplication by components.

To further enhance the representation of spatial and temporal information, two nonlinear encoders M and N are introduced to replace encoders X and b_a in Eq. (14). Then Eq. (14) is updated as

$$H^l = \sigma(W^l H^{l-1} + b^l) \bullet M + N, \quad (15)$$

where the two encoders are denoted by $M = \sigma(W_M X + b_M)$ and $N = \sigma(W_N X + b_N)$, and the weights and biases of the encoders are represented by W_M , W_N and b_M , b_N , respectively. The improved network architecture is illustrated in the orange box in Fig. 2.

The mathematical formulation of the hidden layer in the SI-LB-PINN architecture is defined as follows:

$$\begin{aligned} H^1 &= \sigma(W^1 X + b^1), \\ Z^l &= \sigma(W^l H^{l-1} + b^l), \\ H^l &= (1 - Z^l) \bullet M + Z^l \bullet N, \\ H^L &= \sigma(W^L H^{L-1} + b^L). \end{aligned} \quad (16)$$

Optimizes the thermal-fluid coupling and Improves training stability

Spatial Information & Adaptive Loss Balance PINN

2. The introduction of weight-balancing algorithms

It is assumed that the output of the neural network follows a Gaussian distribution with mean $\hat{u}$ and variance ε_d , where ε_d is also referred to as the uncertainty parameter. The variance is commonly used to quantify the uncertainty of the output. This leads to

$$p(u|\hat{u}(x, t; \theta)) = N(\hat{u}(x, t; \theta), \varepsilon_d^2). \quad (17)$$

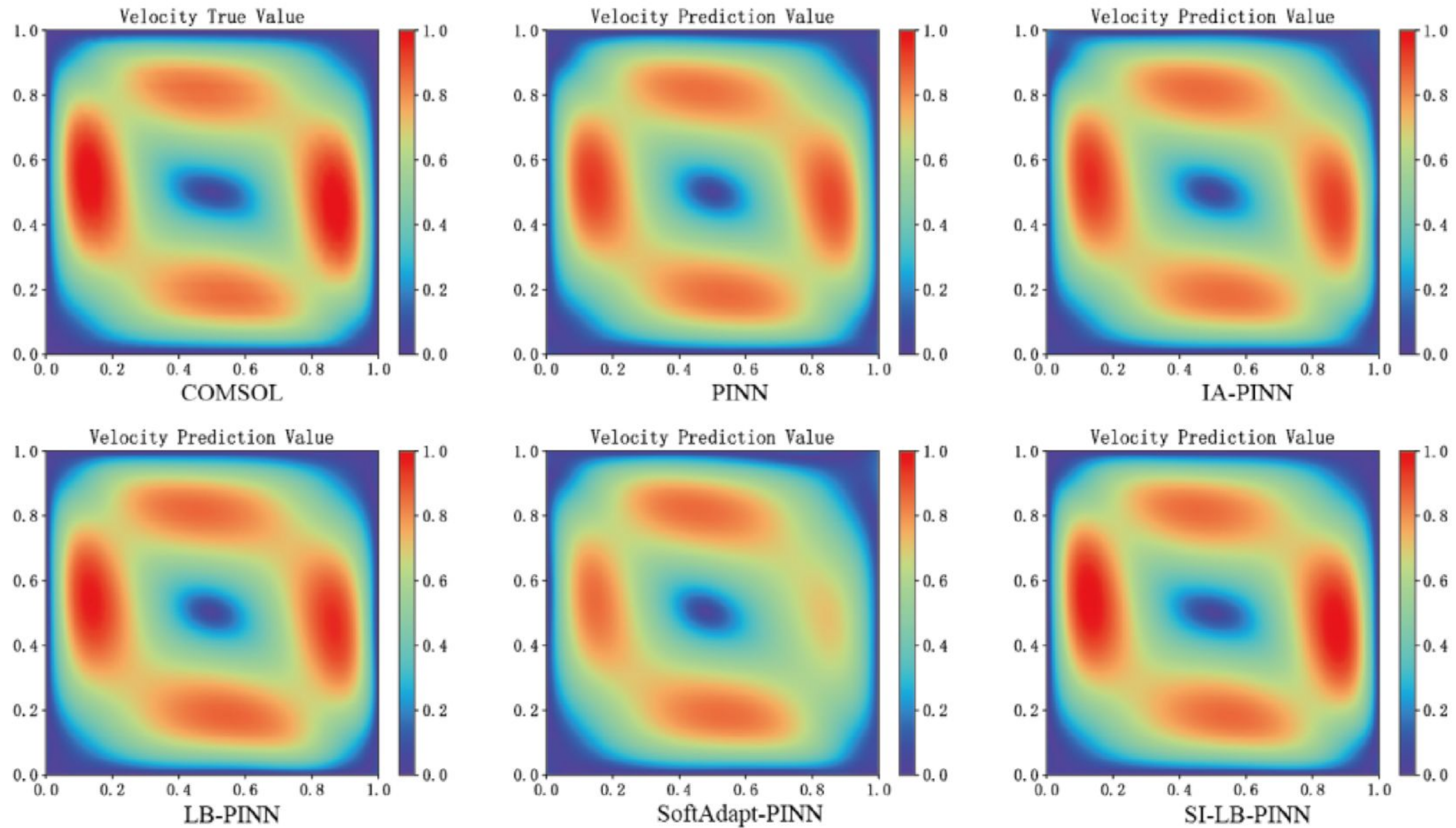
During training, ε_d is typically used as the weight for the corresponding loss term in the loss function. This weight can be optimized through the maximum likelihood function. Based on the minimization goal of the training process, the negative log-likelihood coefficient of the model is minimized. The expression is formulated as follows:

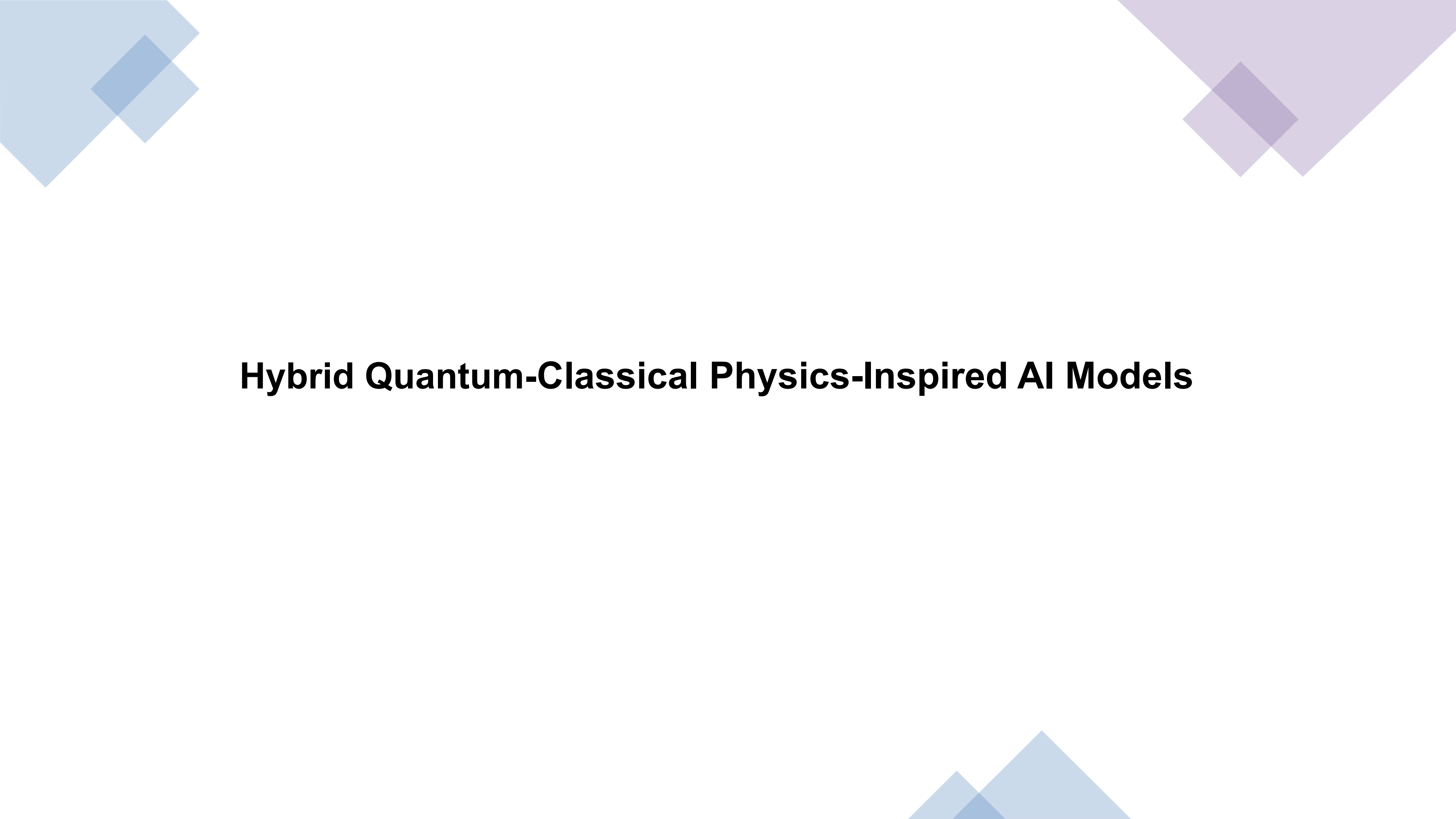
$$\begin{aligned} -\log p(u|\hat{u}(x, t; \theta)) &\propto \frac{1}{2\varepsilon_d^2} \|u - \hat{u}(x, t; \theta)\|^2 + \log \varepsilon_d \\ &= \frac{1}{2\varepsilon_d^2} L_{data}(\theta) + \log \varepsilon_d. \end{aligned} \quad (18)$$

Equation (18) is primarily composed of two components: the residual regression and uncertainty regularization term. In addition, the neural network automatically adjusts the weights of individual loss terms during training by updating the uncertainty parameter ε_d at each iteration, based on maximum likelihood estimation.

Optimizes the thermal-fluid coupling and Improves training stability

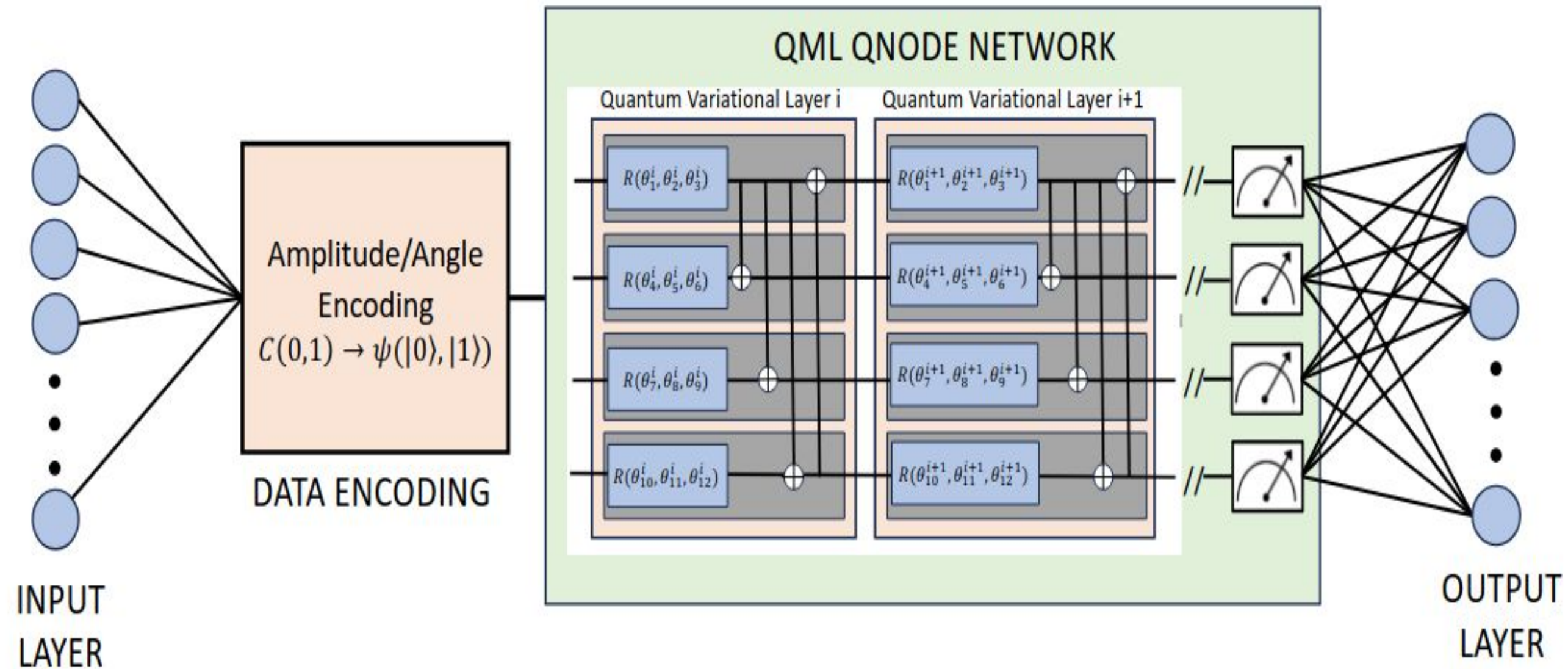
Spatial Information & Adaptive Loss Balance PINN



The image features four decorative geometric shapes in the corners, each composed of overlapping triangles. The top-left and bottom-right shapes are light blue, while the top-right and bottom-left shapes are light purple. The central text is in a bold, black, sans-serif font.

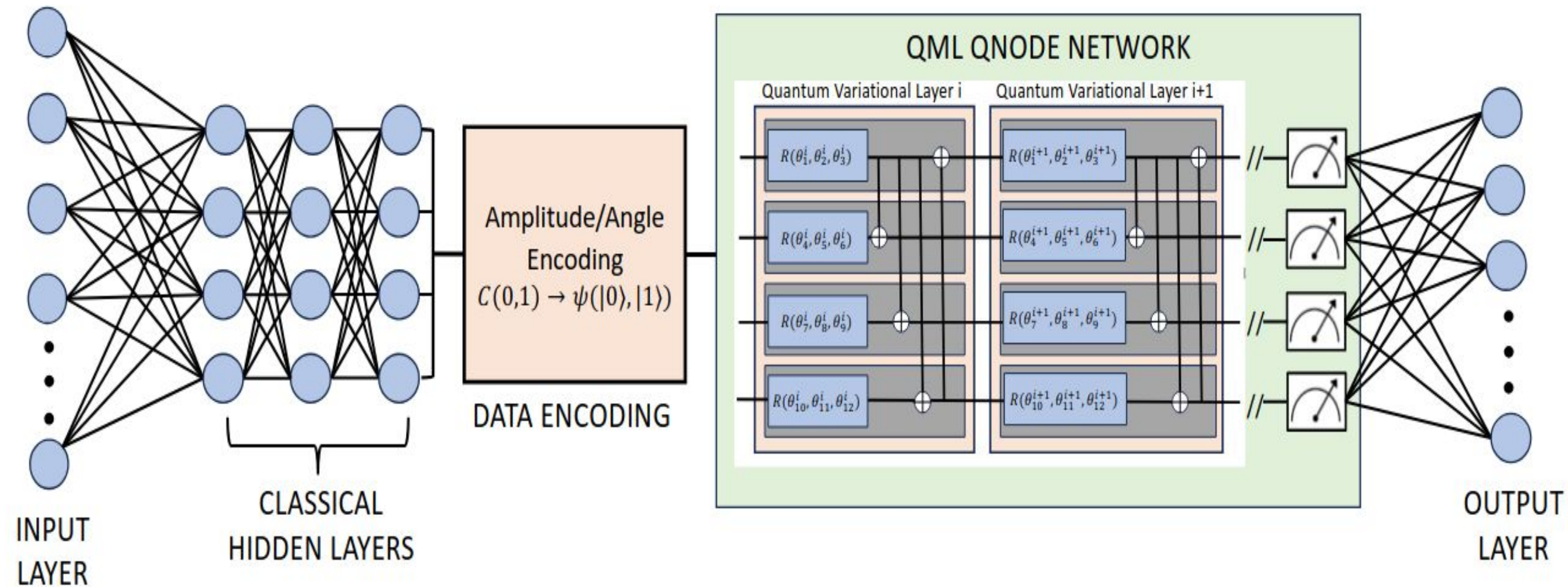
Hybrid Quantum-Classical Physics-Inspired AI Models

Quantum Physics Informed Neural Networks



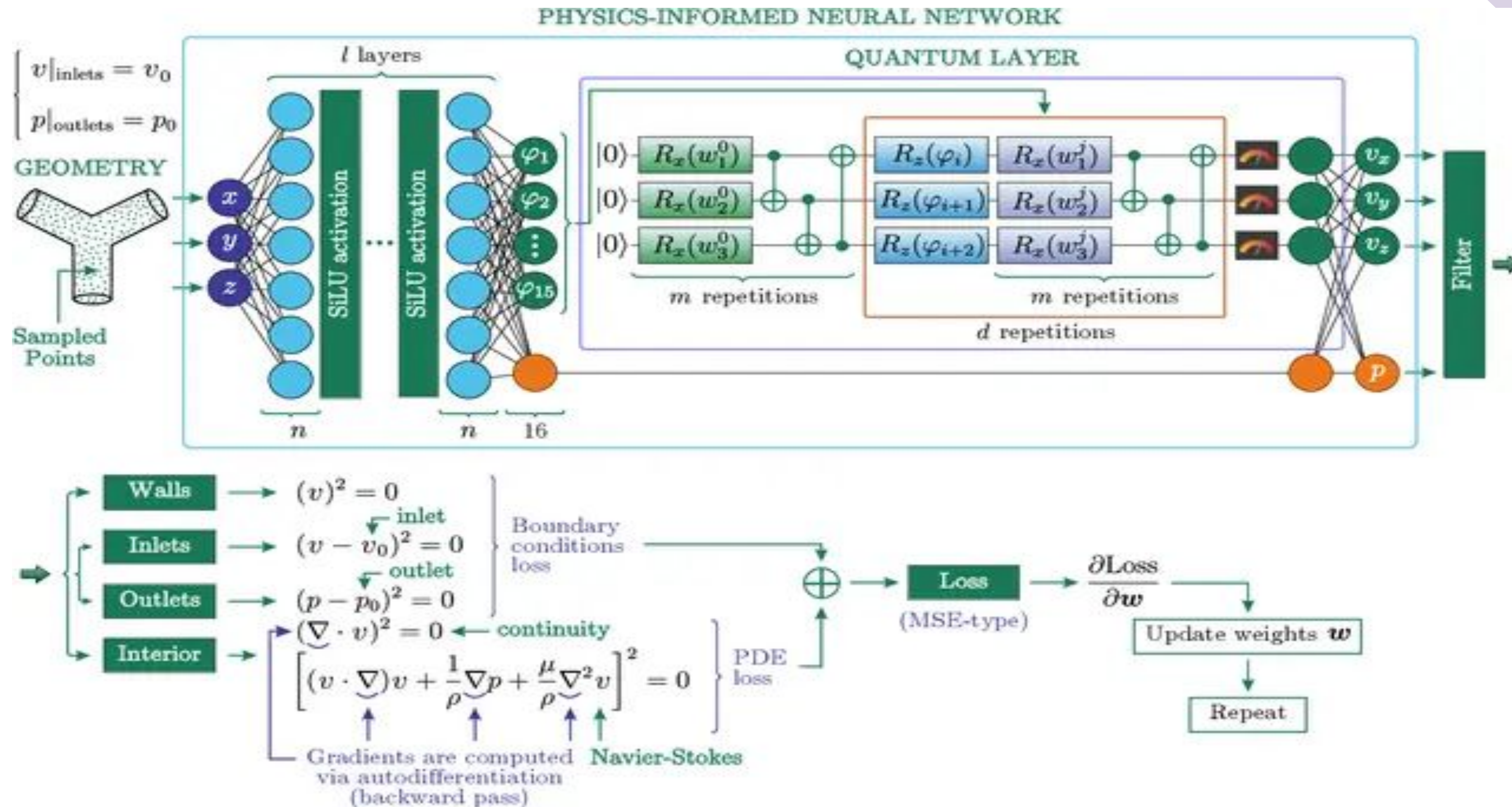
Multi-variational layer quantum network

Quantum Physics Informed Neural Networks



Quantum node layer combined with classical layers

Hybrid Quantum Physics Informed Neural Networks



Variational Quantum Circuit (VQC) with focus on Geometry

Simulation Results

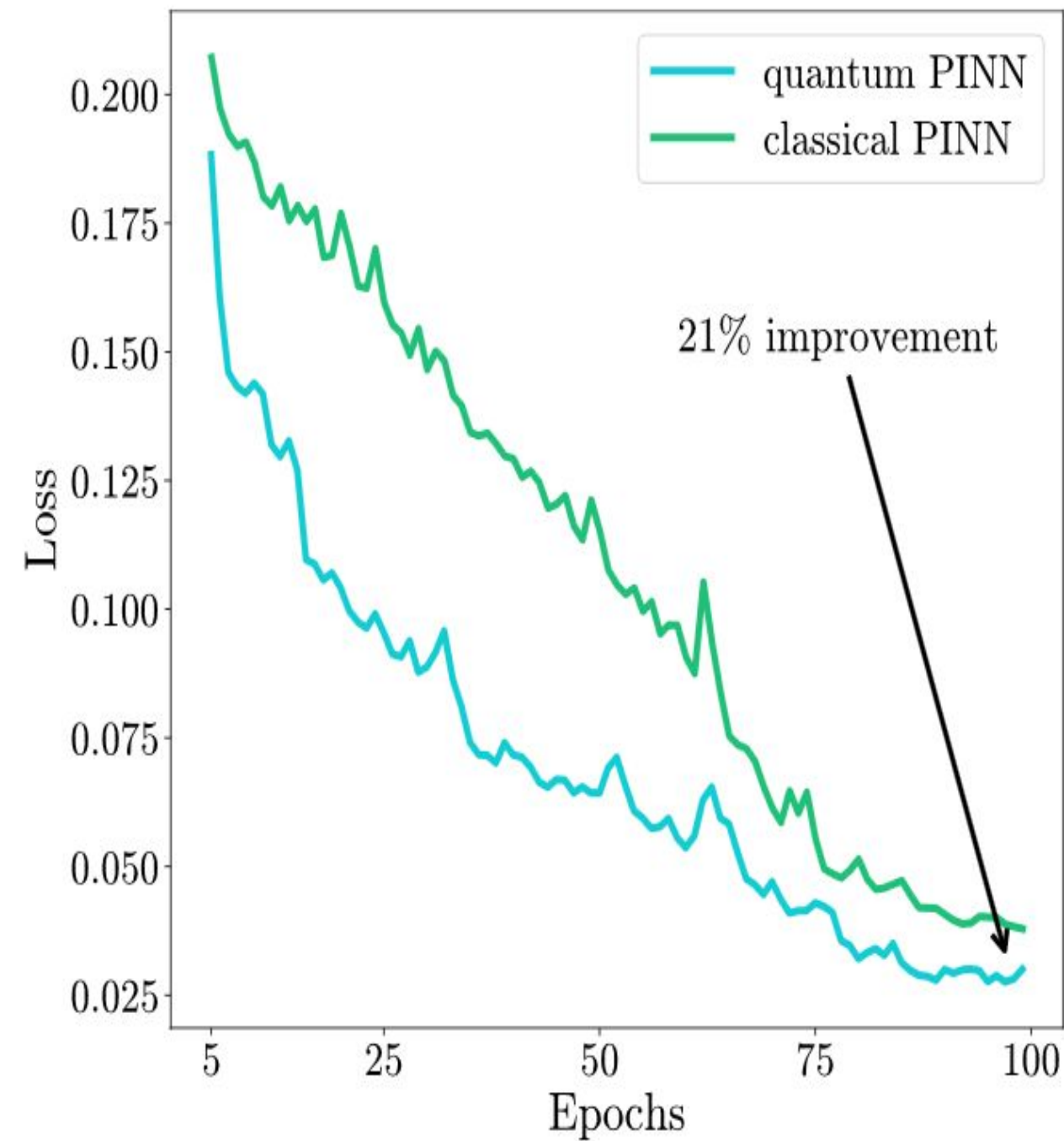
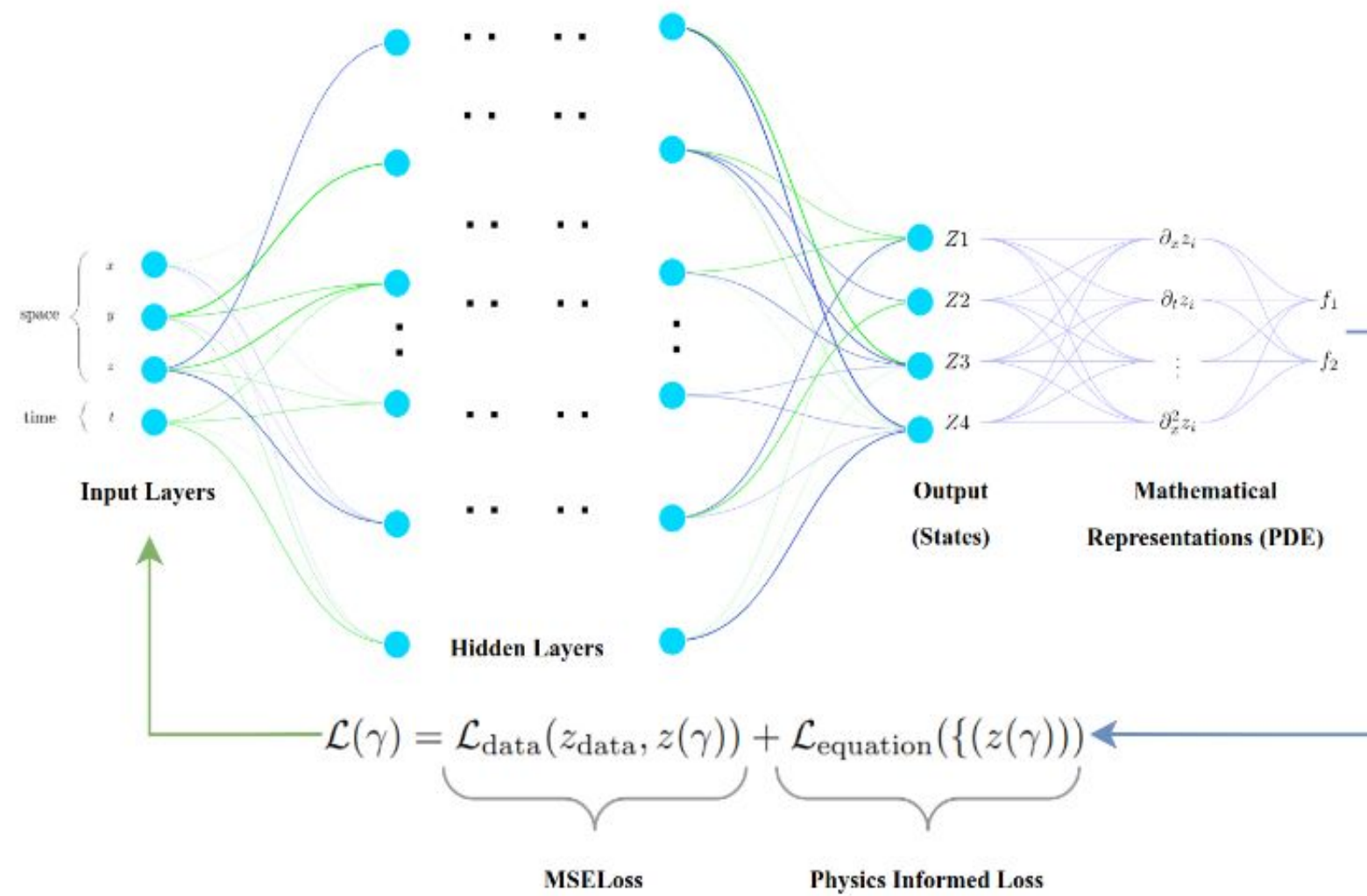
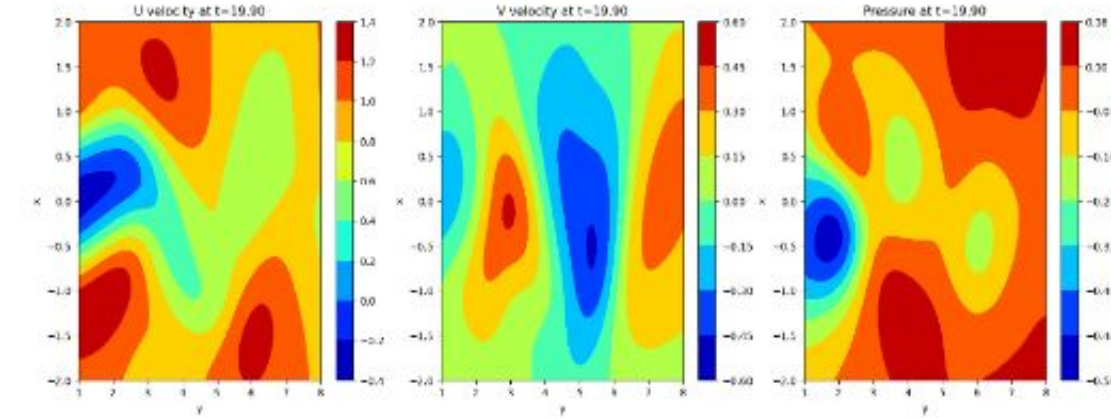


Figure 6. Losses for the classical and hybrid quantum PINNs. HQPINN outperforms the classical PINN in terms of loss value due to higher expressivity.

Attention Enhanced Quantum PINNs



Prediction of current flow by AQ-PINN



Residual between the True & Prediction of current flow

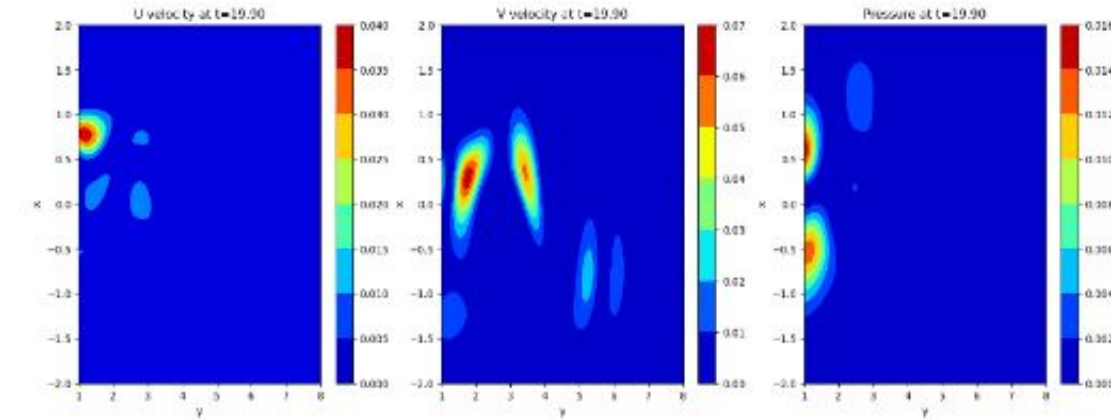


Figure 1: Residuals in the Navier-Stokes equations are computed precisely via **automated differentiation**. The loss function $\mathcal{L}(\gamma)$ is differentiable for both the state variables $z(\gamma)$ and the trainable parameters γ in AQ-PINNs. The predictions of AQ-PINNs can be understood from the diagrammatic representation in the right half.

Simulation Results

The AQ-PINNs architecture consists of several key components, as shown in Fig. 1. The input projection maps the input vector $\mathbf{x} \in \mathbb{R}^3$, representing spatiotemporal coordinates (x, y, t) , into a new space using a linear transformation. The core of AQ-PINNs is the Quantum Multi-head Self-Attention (QMSA) mechanism. This mechanism encodes classical data into quantum states, computes attention scores, and aggregates the results. Each input x_i from the linear layer is encoded into a quantum state using a data loader operator $U^\dagger(x_i)$, where the unitary is represented as an angle embedding followed by circuits such as QMPS, QTTN, or QMERA for different benchmarks [22–25].

For each input x_i , the key K_i , query Q_i , and value V_{ij} are computed as:

$$\begin{aligned} K_i &= \langle x_i | K^\dagger(\theta_K) Z_0 K(\theta_K) | x_i \rangle, & Q_i &= \langle x_i | Q^\dagger(\theta_Q) Z_0 Q(\theta_Q) | x_i \rangle, \\ V_{ij} &= \langle x_i | V^\dagger(\theta_V) Z_j V(\theta_V) | x_i \rangle, \end{aligned} \tag{3}$$

Table 1: Model performance and parameter reduction.

Model	Test Loss Achieved	Decrease in Model Params
Classical A-PINNs	0.0631	-
AQ-PINNs (QMPS)	0.0609	63.29%
AQ-PINNs (QTTN)	0.0593	55.28%
AQ-PINNs (QMERA)	0.0596	51.51%

Matrix Product States (QMPS), Tree Tensor Networks (QTTN),
Multiscale Entanglement Renormalization Ansatz (QMERA)

Hybrid Benchmarks

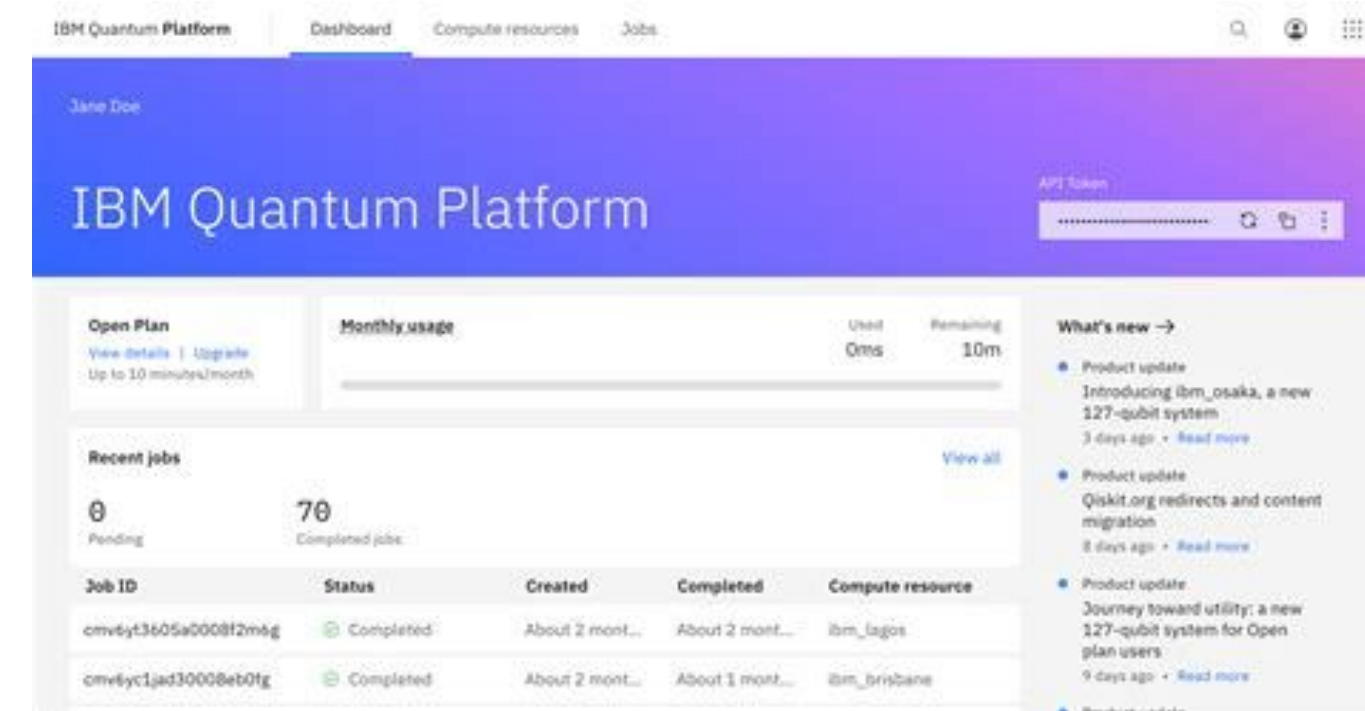
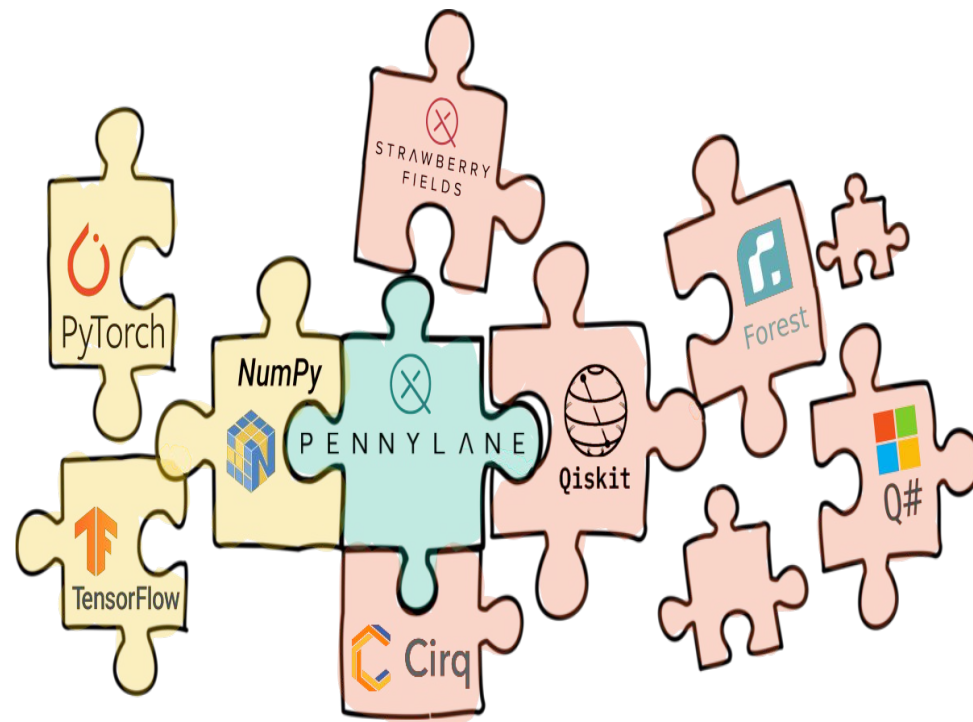
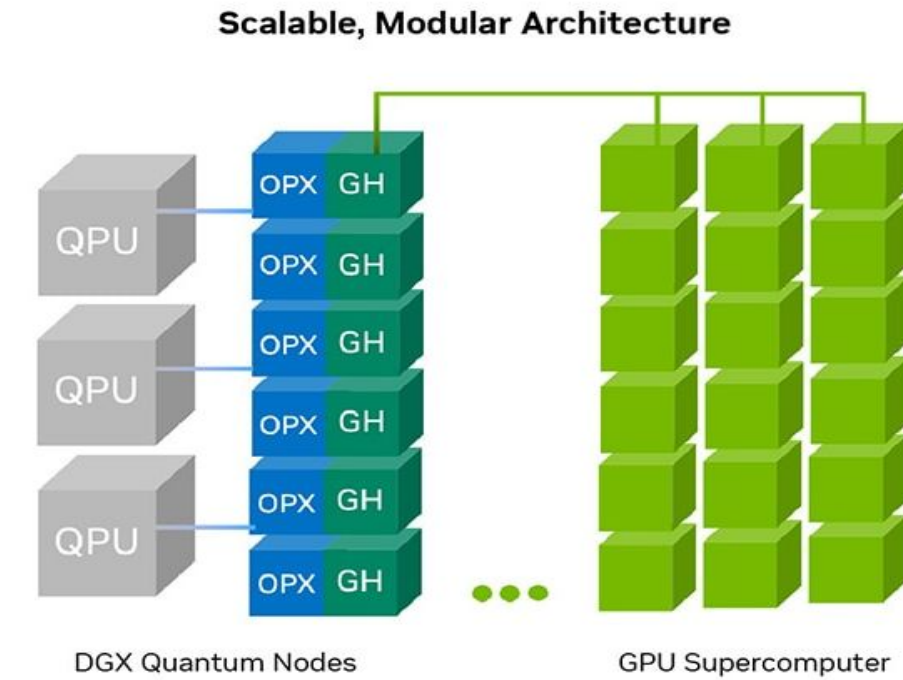
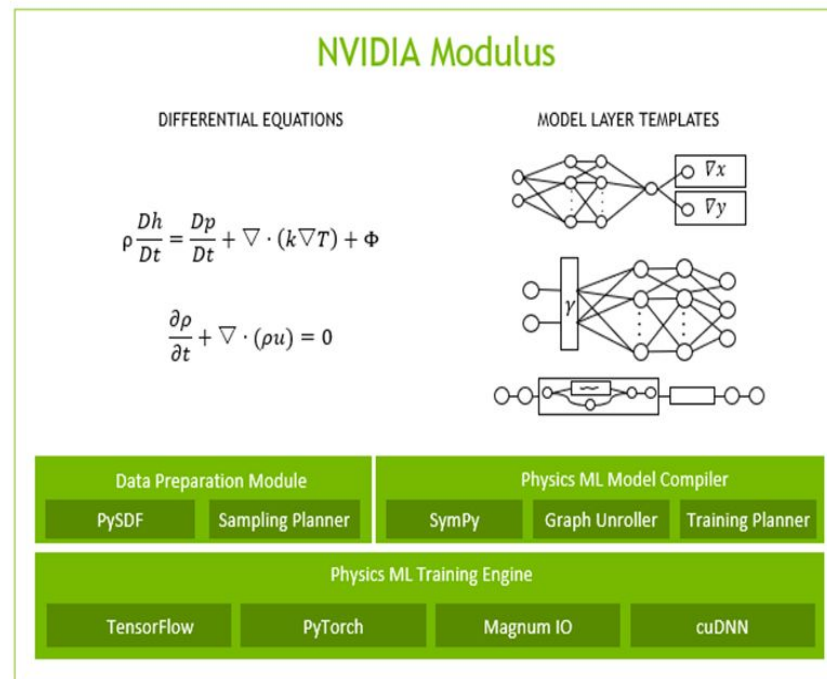
Quantum PINNs

Model	Harmonic		Non-harmonic		Number of trainable parameters
	Accuracy	Trainability	Accuracy	Trainability	
Classical	Medium	High	High	High	High
Quantum	High	Low	Low	Low	Low
Hybrid	Medium–High	Medium–High	Medium	Varies	Medium–High

Table 4: Qualitative assessment of PINN models in the present study.

Qubits: 4–12 qubits for initial benchmarks

Frameworks & Resources



PennyLane, Nvidia Modulus, DGX Quantum, IBM Qantm Credits

Source: <https://pennyLane.ai/> developer.nvidia.com

QC-HPC integration

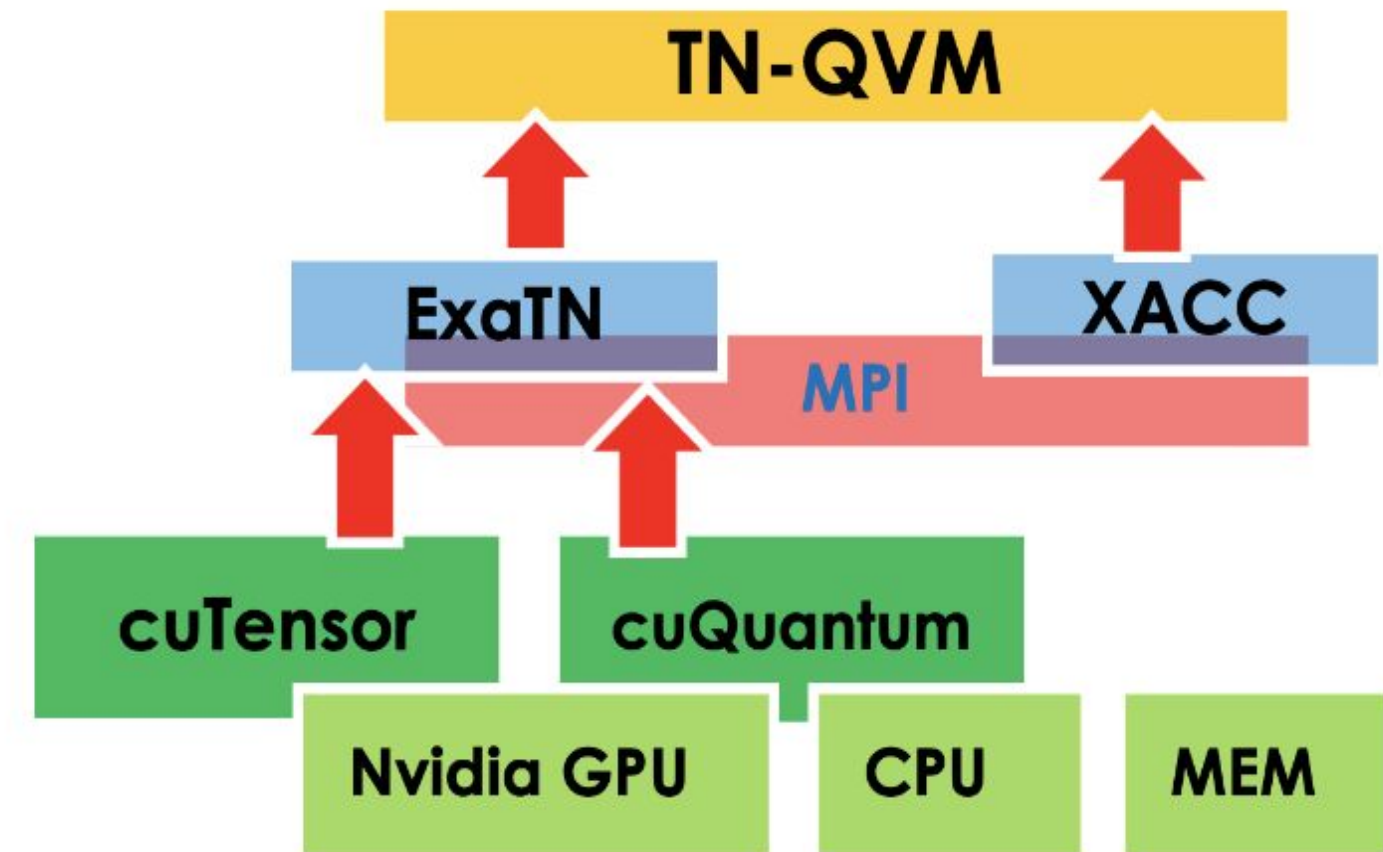
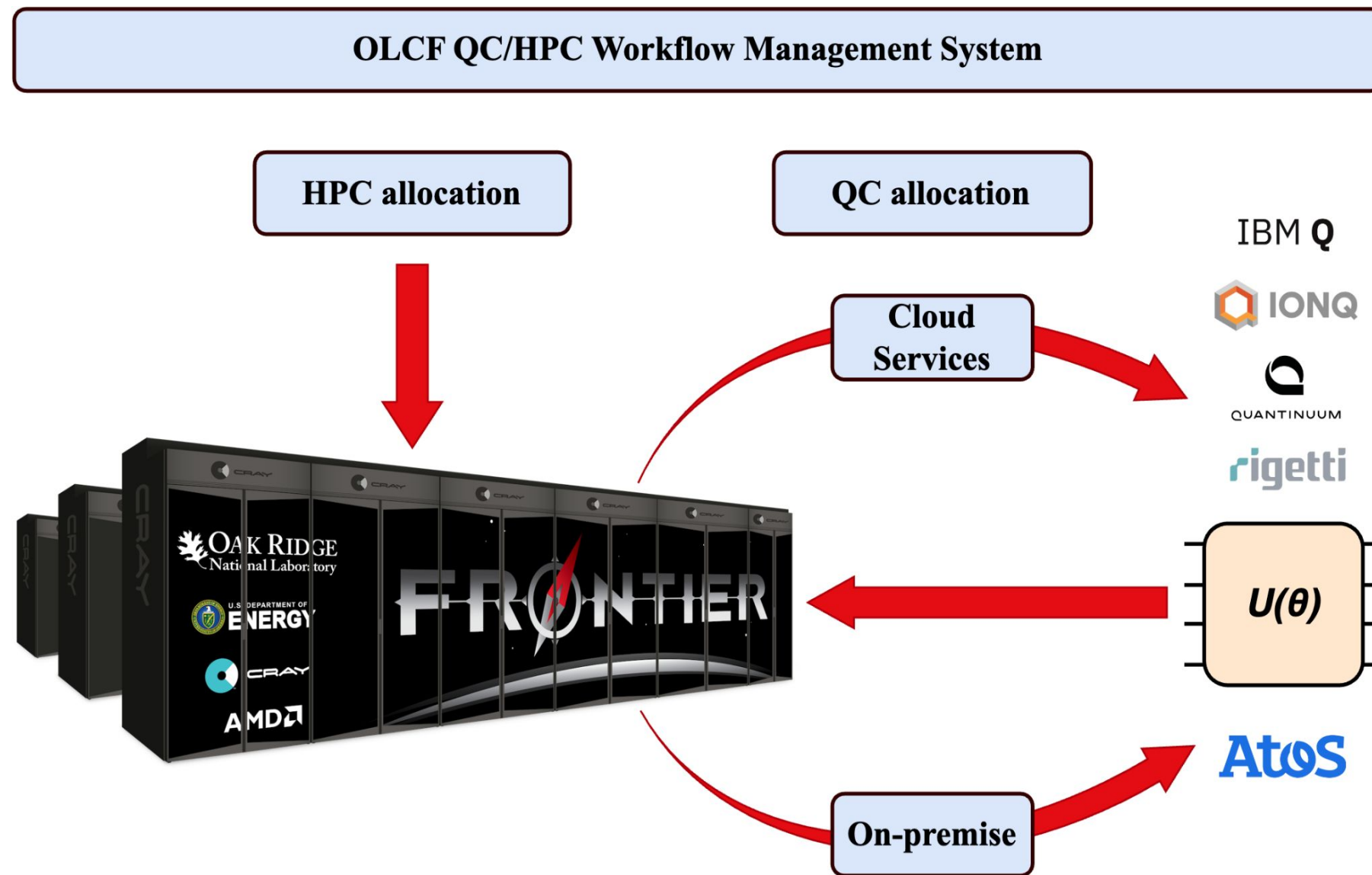


Figure 3: Schematic showing how the TN-QVM, an ORNL-developed tensor-network based accelerator, integrates with various other software packages to form a powerful tool for simulating large circuits and algorithms on HPC platforms.

Outlook

- **Benchmark studies with advanced AI models
(Physics and Geometry Aware)**
- **Hybrid approaches for Quantum advantage**
- **Collaborations with academia and industries**

References

Quantum and Physics-Informed Neural Networks

- Afrah Farea, Saiful Khan, & Mustafa Serdar Celebi. (2025). *QCPINN: Quantum-Classical Physics-Informed Neural Networks for Solving PDEs*. arXiv:2503.16678 [quant-ph]. <https://arxiv.org/abs/2503.16678>
- *Hybrid Quantum PINNs for Computational Fluid Dynamics*. (2025). IOP Science. <https://doi.org/10.1088/2632-2153/ad43b2>
- *Quantum Physics-Informed Machine Learning for Multiscale Simulations*. (2024). arXiv:2403.08954 [quant-ph]. <https://arxiv.org/abs/2403.08954>
- *PINNSFORMER: A Transformer-Based Framework for Physics-Informed Neural Networks*. (2023). arXiv:2307.11833v3 [cs.LG].
- Pengpeng Xiao, Muqing Zheng, Anran Jiao, Xiu Yang, & Lu Lu. (2024). *Quantum DeepONet: Neural Operators Accelerated by Quantum Computing*. [Manuscript].
- Siddhant Dutta, Nouhaila Innan, Sadok Ben Yahia, & Muhammad Shafique. (2024). *AQ-PINNs: Attention-Enhanced Quantum Physics-Informed Neural Networks for Carbon-Efficient Climate Modeling*. arXiv:2409.01626v1 [cs.LG].

Physics-Informed Neural Networks in Crystal Growth Simulations

- D. Kolberg, C. Anders, & S. Reke. (2023). *Fast Prediction of Transport Structures in the Melt by Physics-Informed Neural Networks during ‘VMCz’ Crystal Growth of Silicon*. *Journal of the Institute of Metals*, <https://doi.org/10.1080/00219592.2023.2236656>
- J. Ling, Y. Zhang, & D. Chen. (2023). *Simulation of Thermal-Fluid Coupling in Silicon Single Crystal Growth Based on Gradient Normalized Physics-Informed Neural Network*. *Physics of Fluids*. <https://doi.org/10.1063/5.0123811>
(Code repository: <https://github.com/callmedrcom/SIPINN>)
- Z. Wang, F. Chen, & M. Li. (2024). *RF-PINNs: Reactive Flow Physics-Informed Neural Networks for Field Reconstruction of Laminar and Turbulent Flames*. *Journal of Computational Physics*, 113698. <https://doi.org/10.1016/j.jcp.2024.113698>
- Y. Zhao, H. Liu, & J. Xu. (2025). *Research on the Thermal-Fluid Coupling in the Growth Process of Czochralski Silicon Single Crystals Based on an Improved Physics-Informed Neural Network*. *AIP Advances*, 15(10), 105202. <https://doi.org/10.1063/5.0271778>

Thank You

Name: A Seshaditya

Email: aditya@quasi.digital

Websites: <https://quasi.digital>

<https://github.com/adytiaa/quasi.ai>

<https://adytiaa.github.io/quasi.ai/>



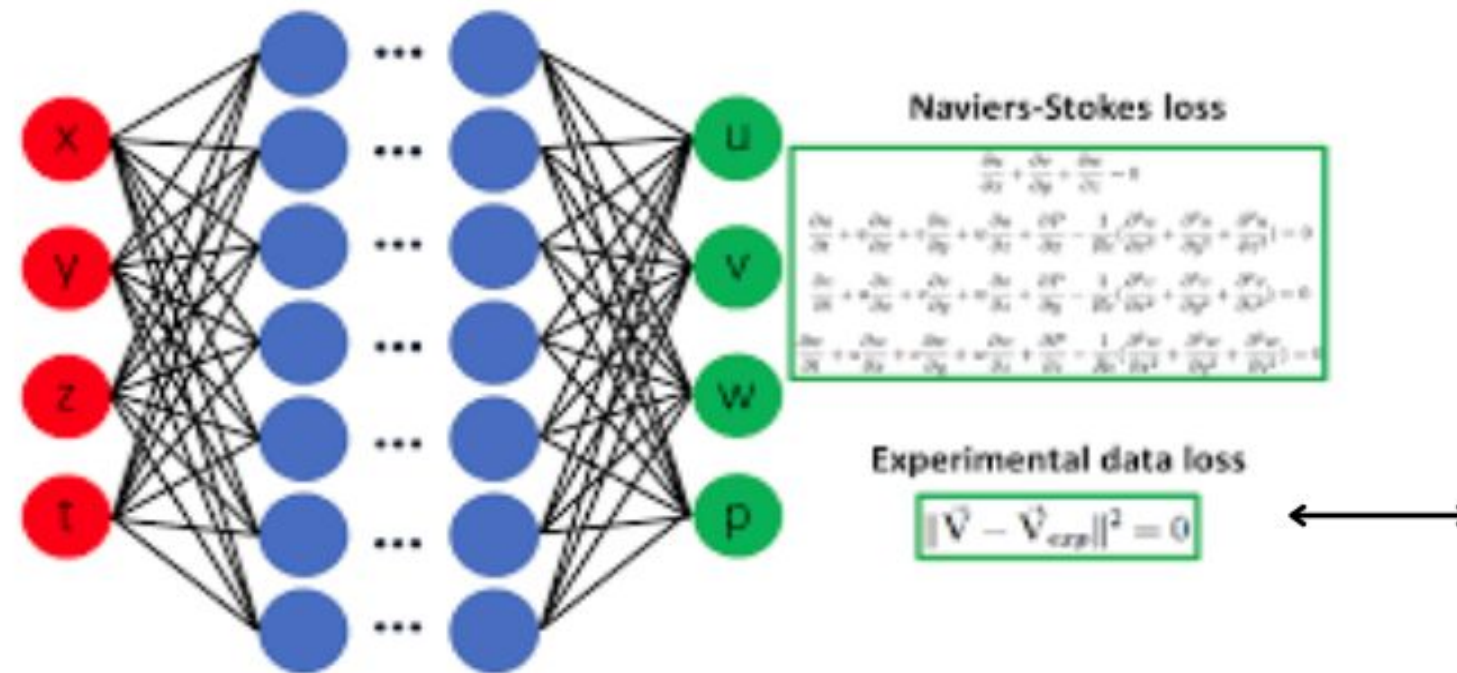
Classical Benchmarks

PINNs, Neural Operators, NeuralDEM, Universal Physics Transformer Models

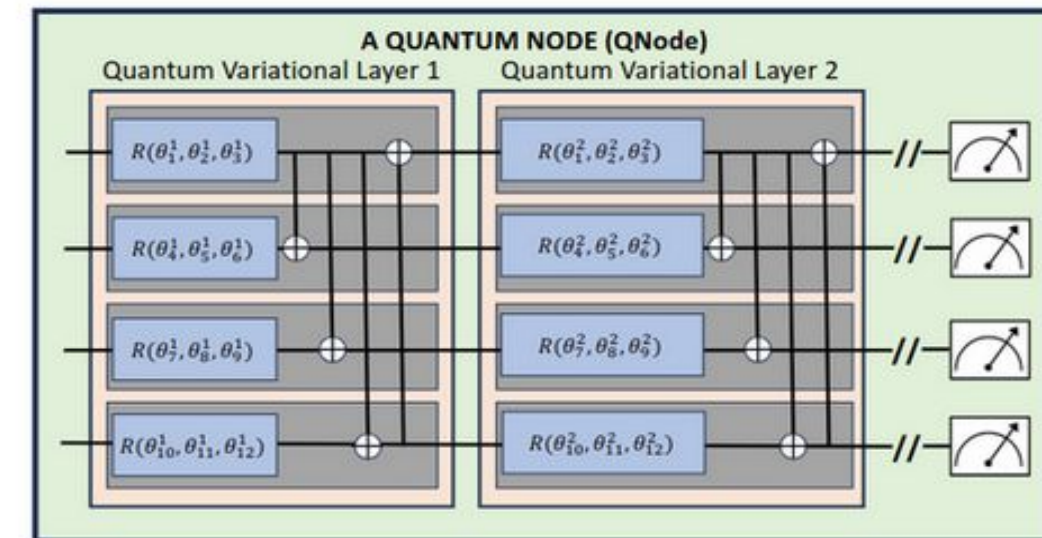
	ShapeNet-Car		AhmedML			DrivAerML			Neural field	Mesh independent
	p_s	u	p_s	u	ω	p_s	u	ω		
PointNet	12.01	3.05	8.51	5.43	67.29	23.26	28.12	880.0	✗	✗
GRAPH U-NET	10.33	2.48	6.42	4.14	53.53	16.13	17.98	330.0	✗	✗
GINO	13.27	2.52	7.90	6.23	71.80	13.03	40.58	98.5	✓	✓
LNO	9.05	2.29	12.91	7.56	72.04	20.51	23.27	173.0	✓	✓
UPT	6.41	1.49	4.25	2.73	15.03	7.44	8.74	90.2	✓	✓
OFormer	7.05	1.60	3.55	2.22	7.78	4.48	6.64	60.4	✓	✓
Transolver	6.46	1.62	3.45	2.05	8.22	4.81	6.78	38.4	✗	✗
Transformer	4.86	1.17	3.41	2.09	6.76	4.35	6.21	47.9	✗	✗
AB-UPT	4.81	1.16	3.01	1.90	6.52	3.82	5.93	35.1	✓	✓

Overview

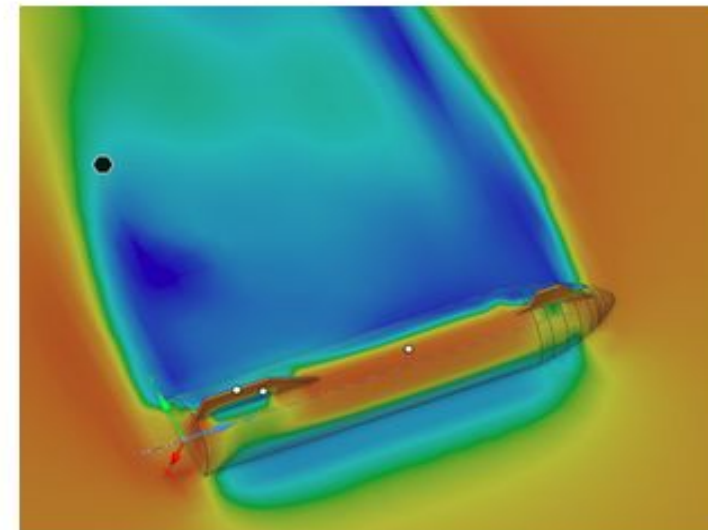
Classical PINN



QML Quantum Node Layers

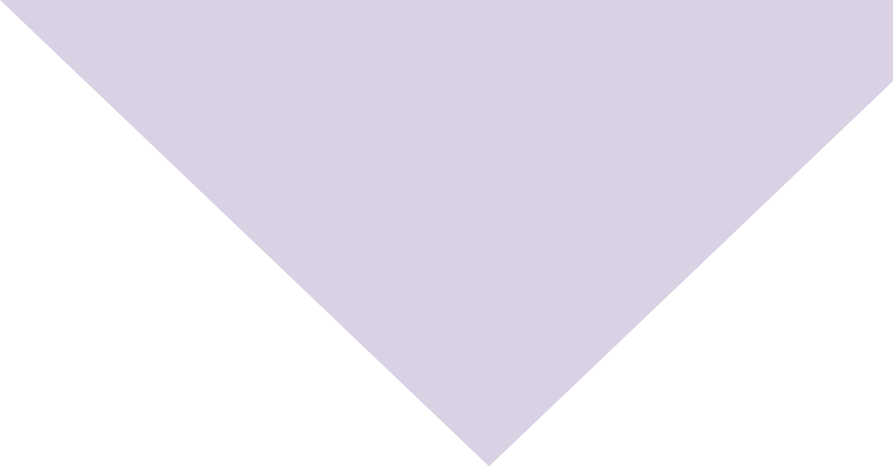


Turbulent
Flows



CRFP
Vessels

Quantum advantage to Classical PINNs



Role of PINNs

Perform Parameter Sensitivity Analysis

- 1. Temperature gradients**
 - 2. Precursor pressure concentrations**
 - 3. Flow rates**
 - 4. Substrate properties**
- 

Role of PINNs

PINN methodology

- **Modified Loss Function:**

$$L_{\text{total}} = L_{\text{PDE}} + L_{\text{BC}} + L_{\text{IC}} + \lambda L_{\text{SAL}}$$

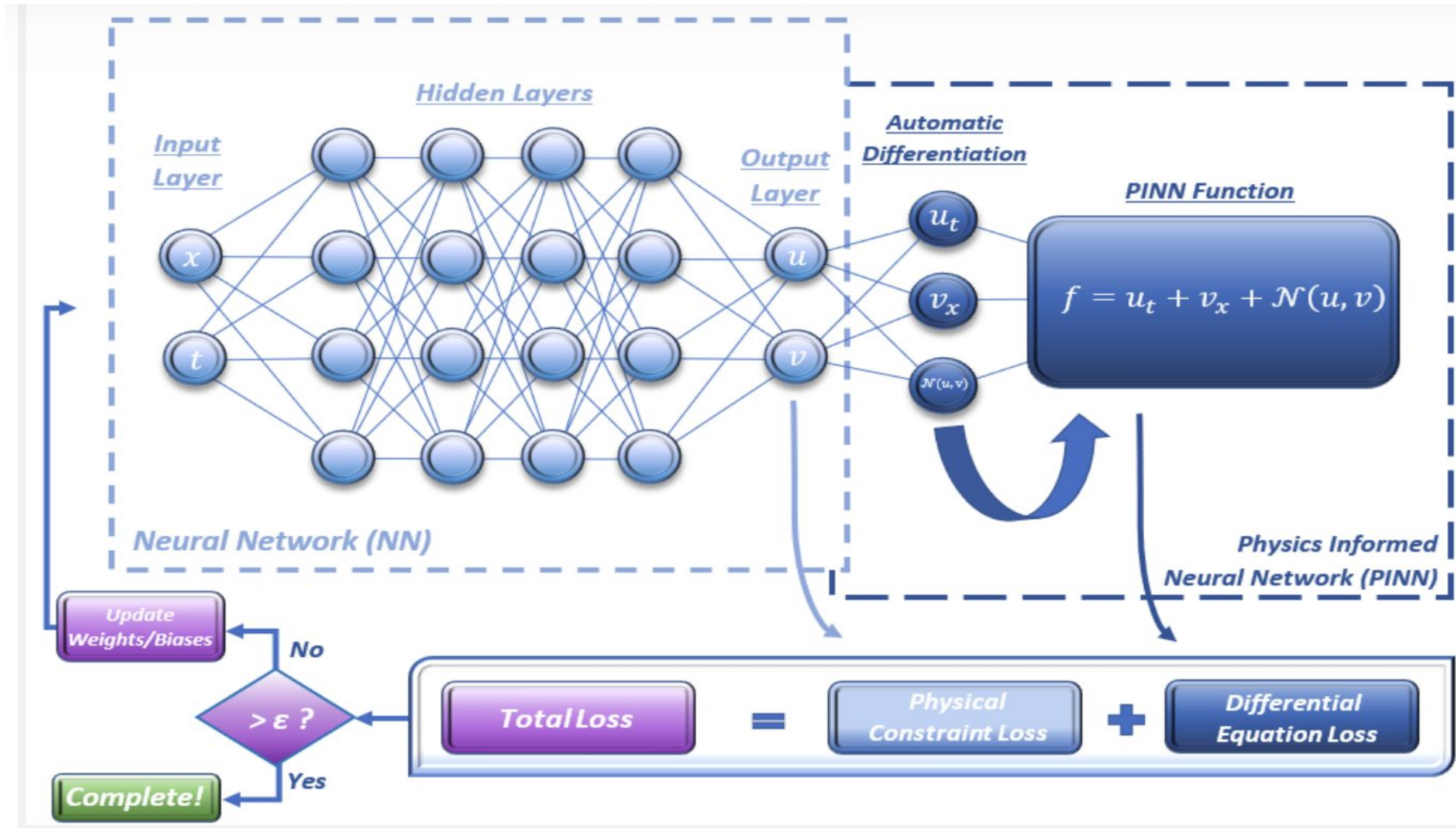
- L_{PDE} is the physics-informed loss
- L_{BC} and L_{IC} are boundary and initial condition losses
- L_{SA} is the sensitivity analysis term
- λ is a weighting factor

Computation done with automated differentiation

Overview

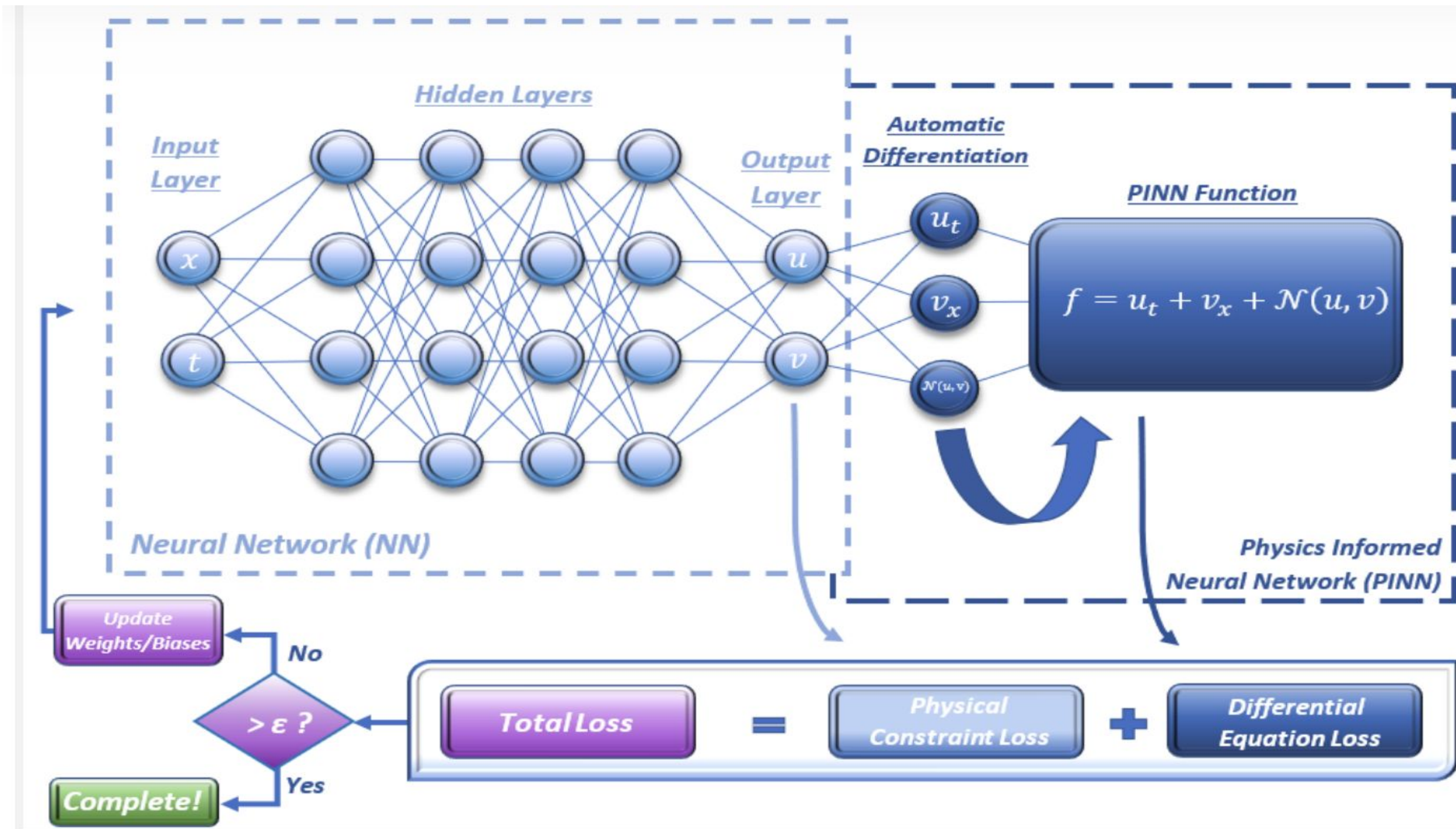
- **Combining Quantum with Physics-Inspired AI models for accelerating Computational Fluid Dynamic (CFD) Simulations**
- **Addressing the problems associated with CFD**
 - **High Computational Cost**
 - **Expensive High Fidelity Simulations**

PINNs: combining NN with Physics



Higher Accuracy & Multi-Physics Capability with Less Data

PINNs: combining NN with Physics



Higher Accuracy & Multi-Physics Capability with Less Data

Unified PDE Solver (UPS)

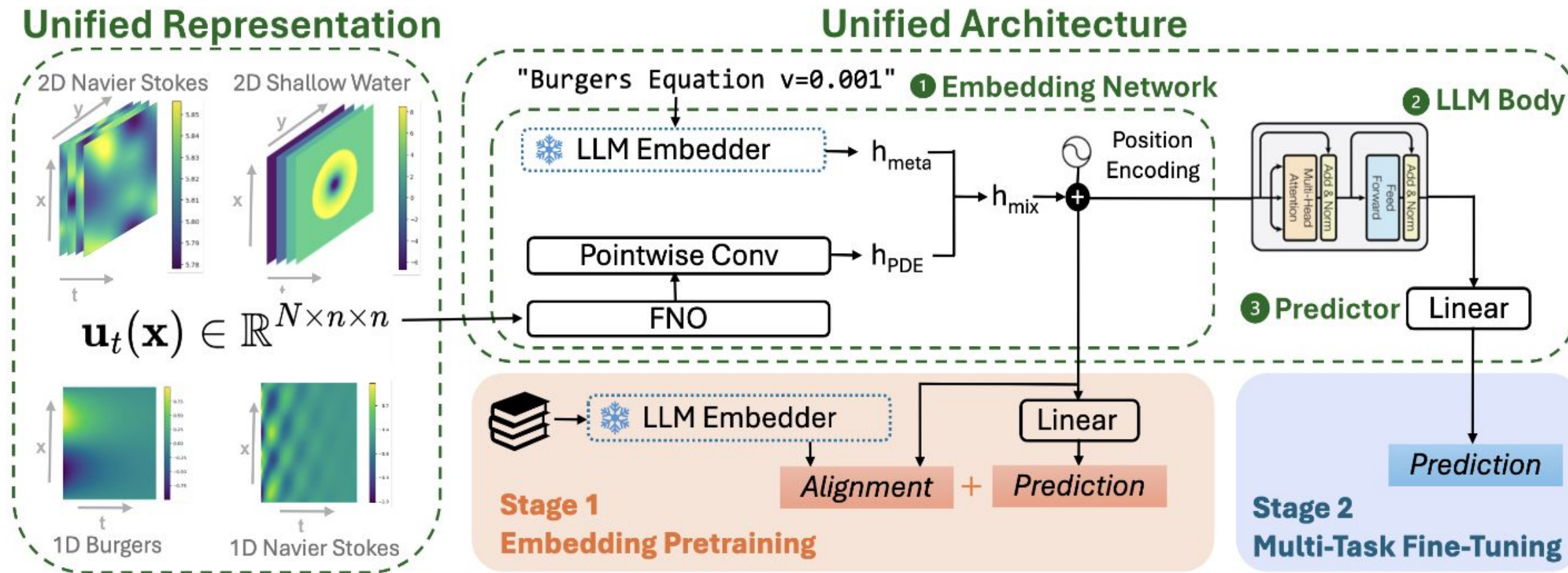


Figure 1: To adapt pretrained LLMs for PDE solving, UPS first transforms PDE of different dimensions, channels, and resolutions into a *unified representation* (left panel). Then, the data is processed with a *unified architecture* that integrates FNO layers, PDE metadata, and LLMs (right panel). The architecture is trained in two stages. In stage 1, we pretrain the embedding network using a joint loss that simultaneously optimizes (i) the distribution similarity between PDE features and text embeddings to align the modalities, and (ii) the prediction performance of extracted PDE features. In stage 2, we fine-tune the entire model on a dataset that combines multiple families of spatiotemporal PDEs with varying domain dimensions, initial/boundary conditions, and coefficients. UPS achieves competitive results with significantly better sample-efficiency than existing methods.

Introduction



A Seshaditya



Academic Experience:

Doctoral Thesis in Density Functional Theory

(TU Berlin)

Dimensionality Behaviour of Kinetic Functionals

(FU Berlin)

Industry Experience:

Data Science and Machine Learning

BMBF Projects KILIDAR, KIKOSA

(Ancud IT)

Physics Informed Quantum Neural Networks

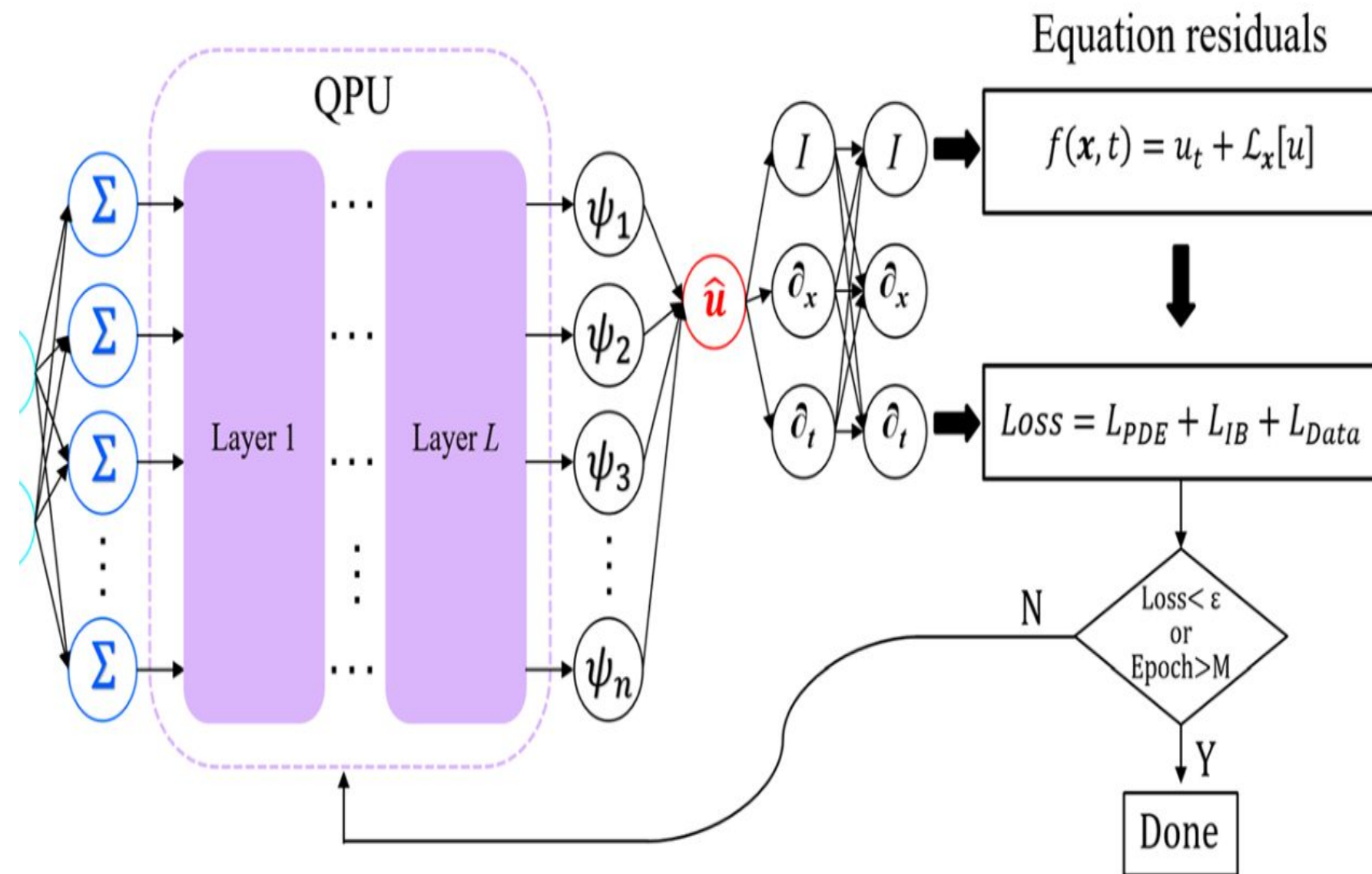
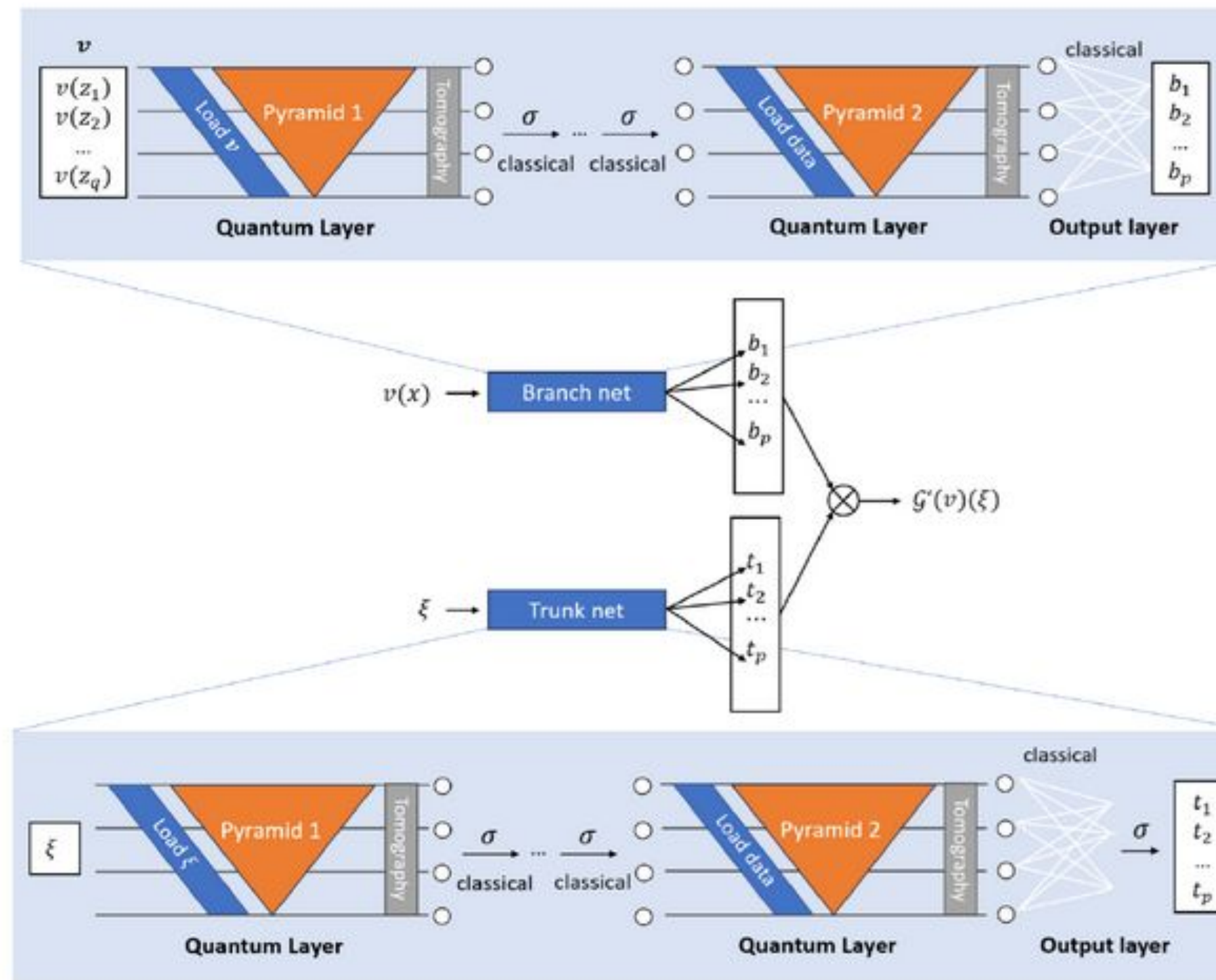


FIG. 4. Schematic diagram of PI-QNN.

Quantum DeepONets



Variational Quantum Circuit (VQC) with focus on Geometry

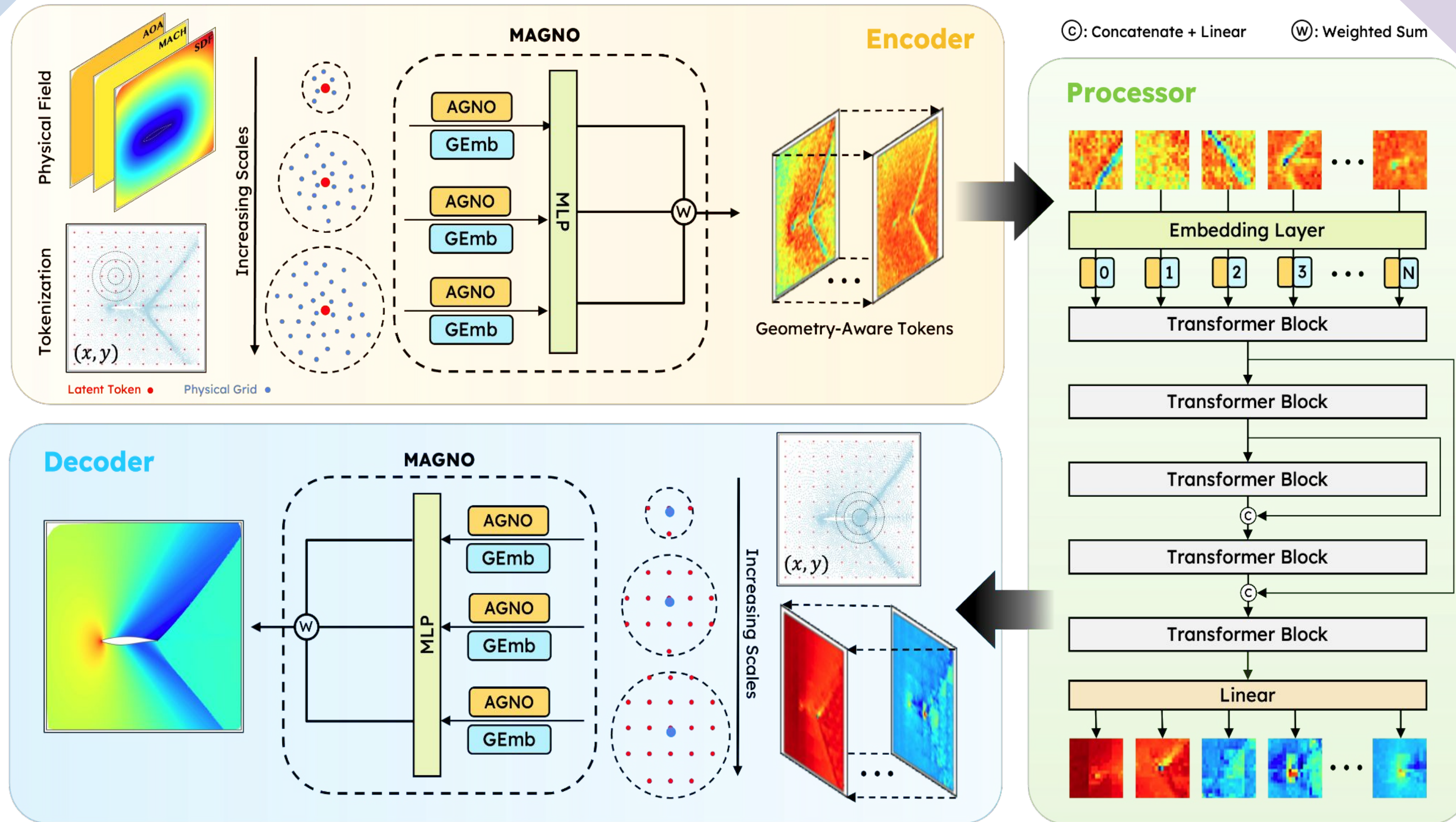
GAOT: Geometry Aware Operator Transformers

The GAOT (Geometry-Aware Operator Transformer) architecture for CFD simulations integrates geometric understanding with transformer capabilities.

1. **Geometric Encoder:** Processes input geometry (e.g., point clouds, SDFs) to extract relevant features.
2. **Transformer Blocks:** Utilize self-attention to learn long-range dependencies and interactions within the geometric features.
3. **Flow Decoder:** Transforms the learned features into the desired fluid flow field (e.g., velocity, pressure) at query points.

In essence, it learns a mapping from a given geometry to its corresponding flow field, enabling rapid, geometry-aware predictions after training on CFD simulation data.

GAOT: Geometry Aware Operator Transformers



Hybrid Quantum Physics Informed Neural Networks

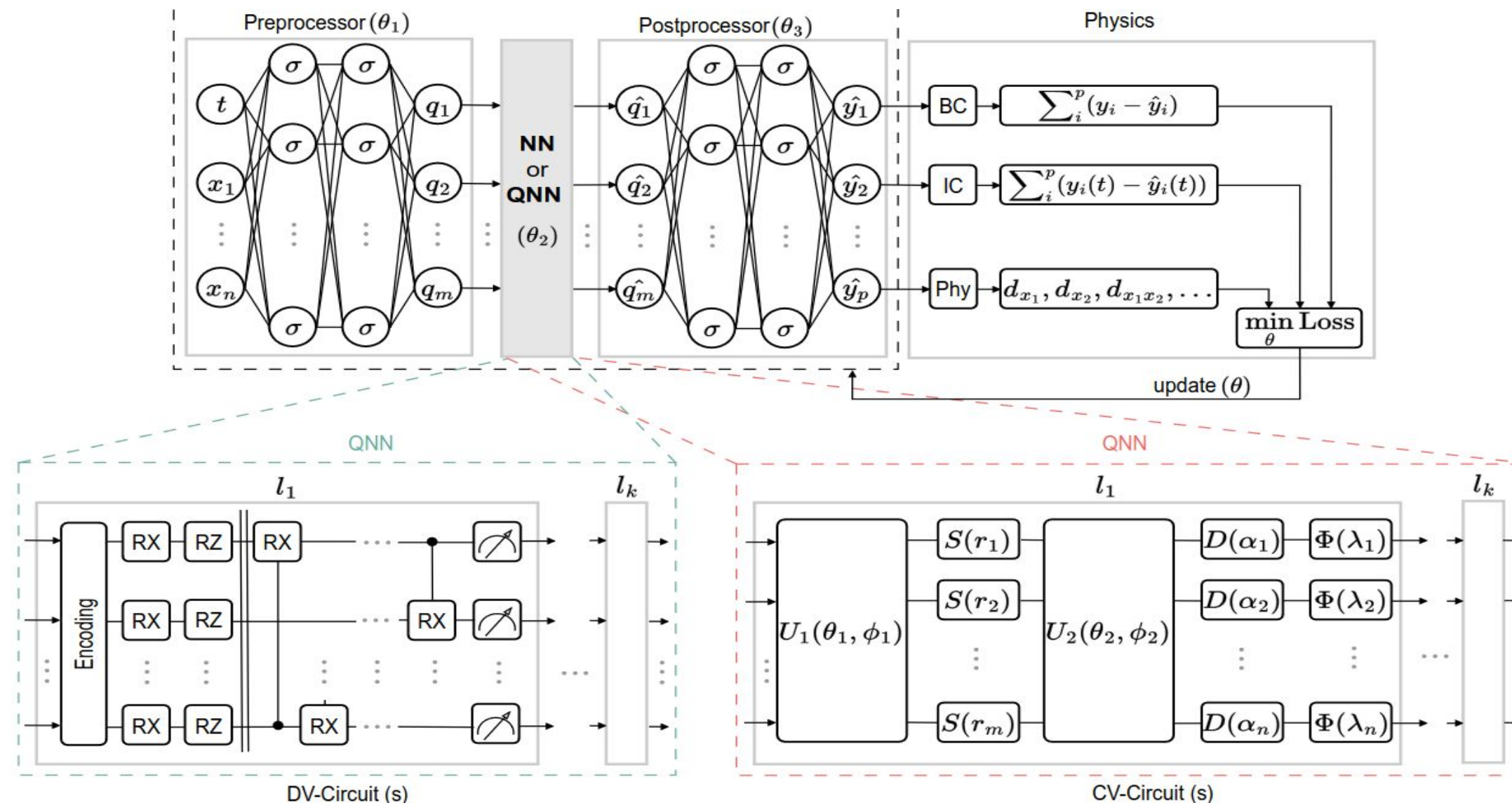


Figure 1: Architecture of the quantum-classical physics-informed neural network (QCPINN) framework. The figure illustrates the hybrid structure: a classical preprocessor encodes the inputs, which are then fed into the QNN or classical neural network (NN). The QNN is implemented as either CV-circuit or DV-circuit with k layers. A classical postprocessor then decodes the QNN or NN outputs. The integrated loss function concurrently minimizes PDE residuals and initial/boundary condition errors, ensuring the model adheres to physical constraints throughout training.

Variational quantum circuit (VQC) with focus on Geometry

Physics Informed Quantum Neural Networks

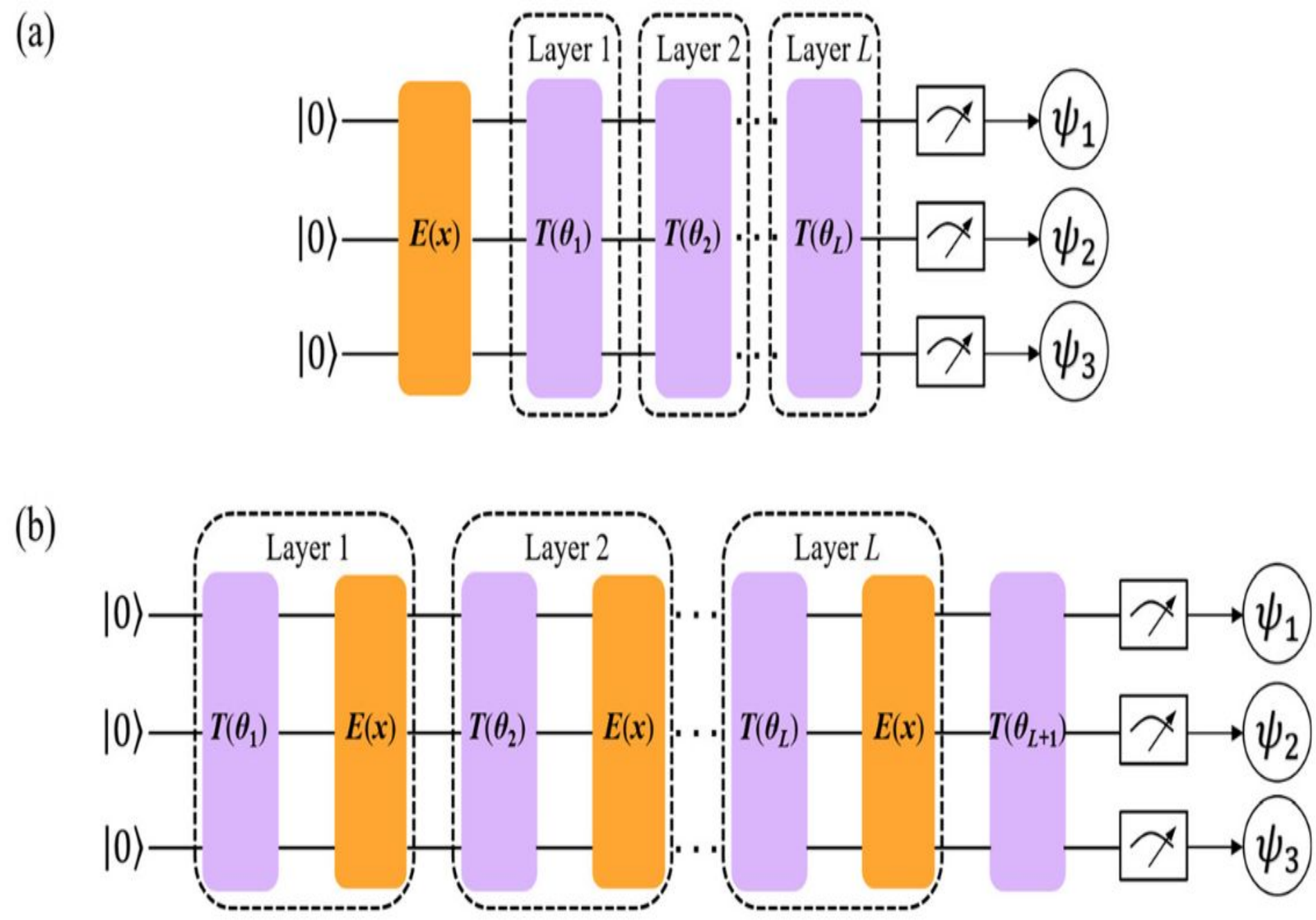
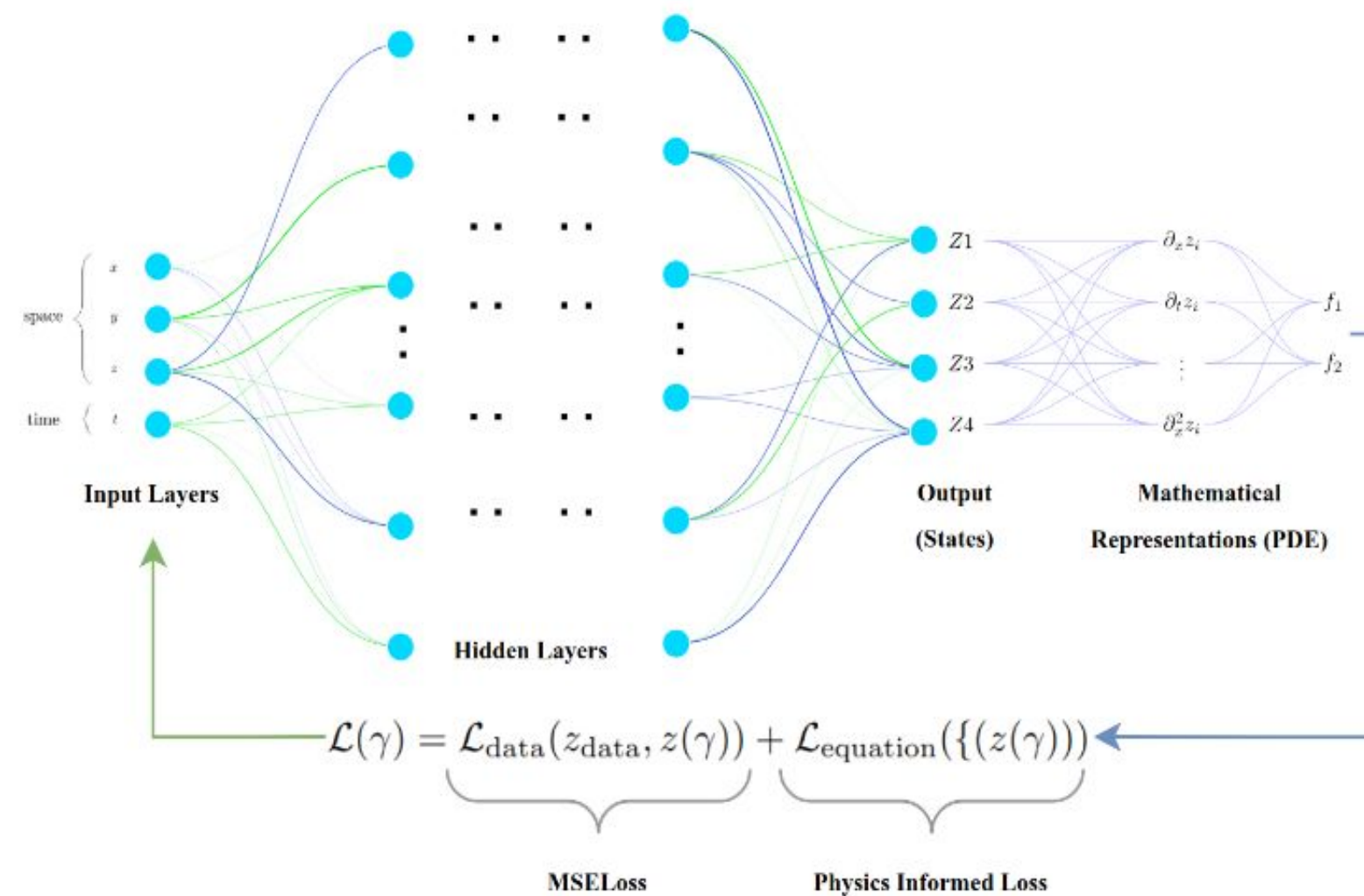
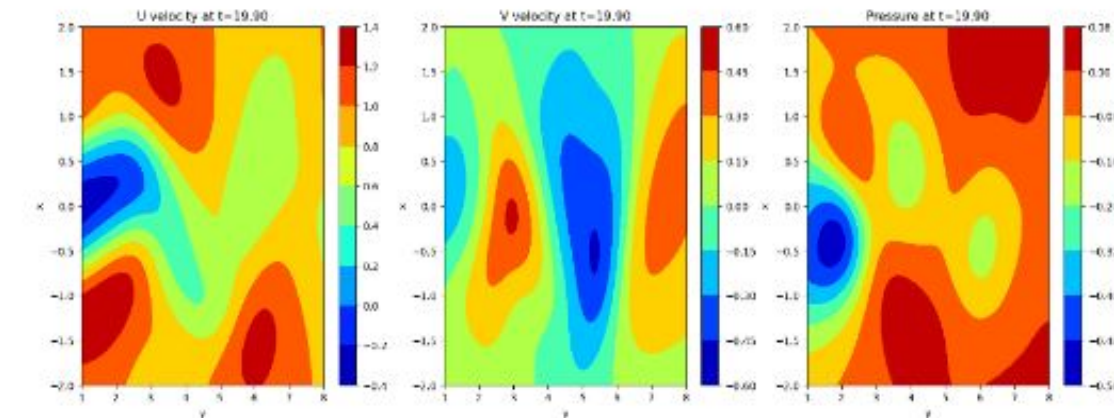


FIG. 3. Schematic diagram of (a) QNN-I and (b) QNN-II with three qubits.

Advanced Quantum PINNs-Attention Enhanced



Prediction of current flow by AQ-PINN



Residual between the True & Prediction of current flow

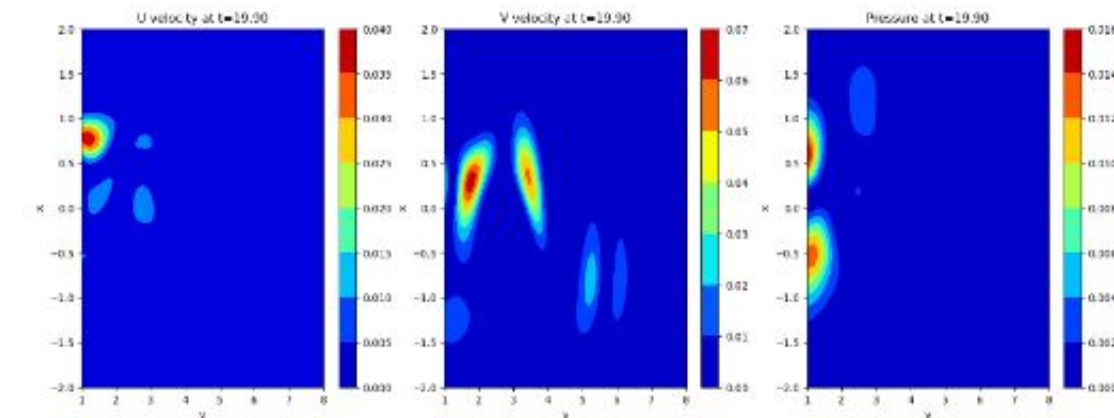
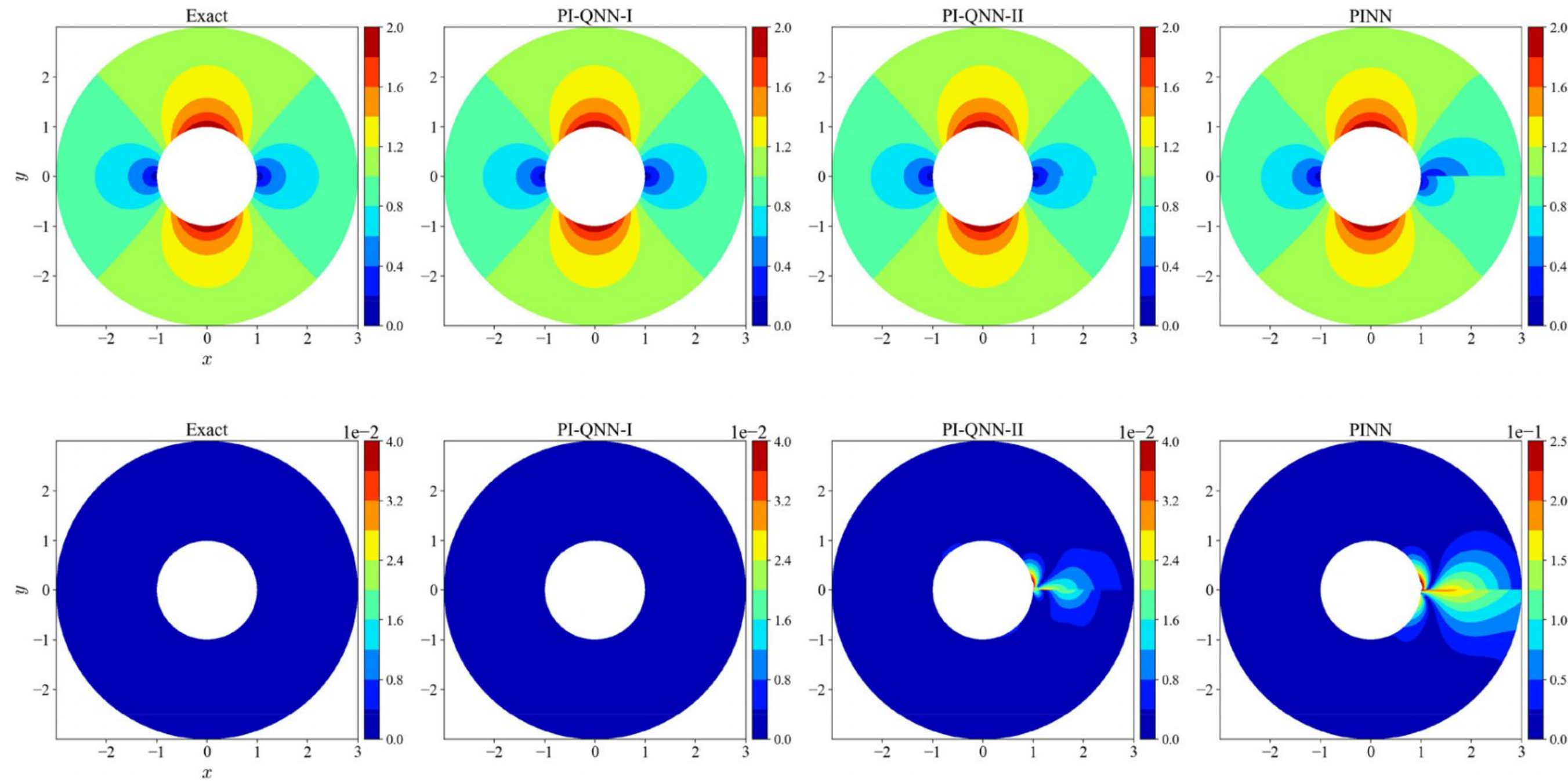


Figure 1: Residuals in the Navier-Stokes equations are computed precisely via **automated differentiation**. The loss function $\mathcal{L}(\gamma)$ is differentiable for both the state variables $z(\gamma)$ and the trainable parameters γ in AQ-PINNs. The predictions of AQ-PINNs can be understood from the diagrammatic representation in the right half.

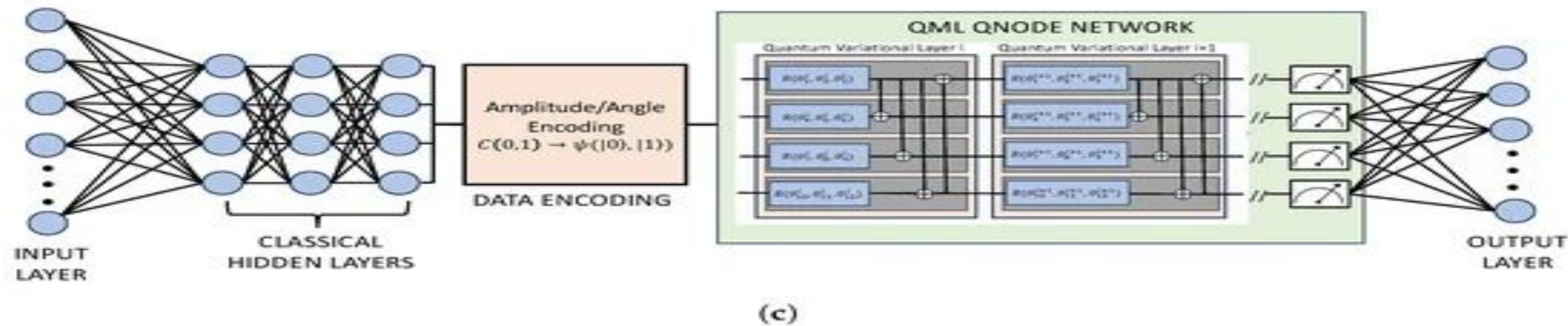
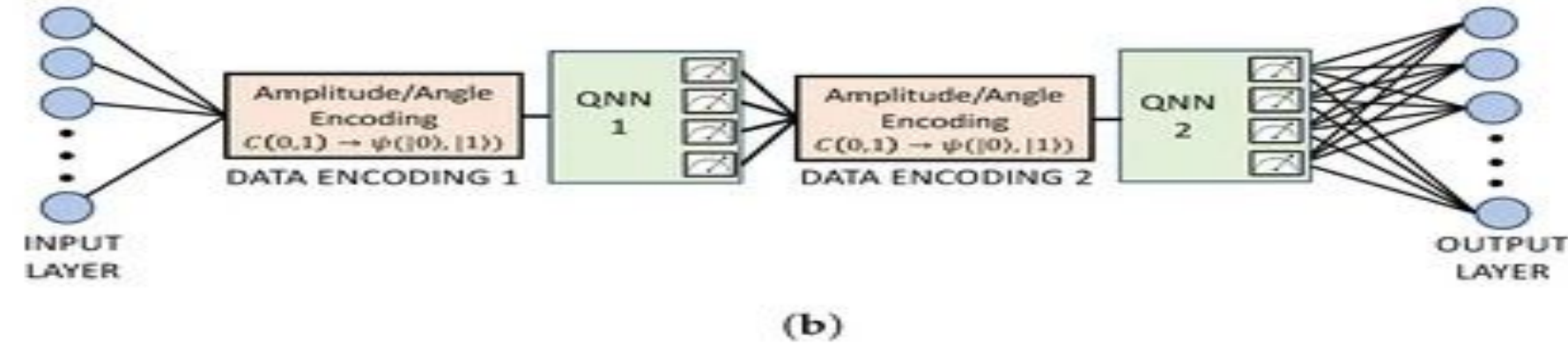
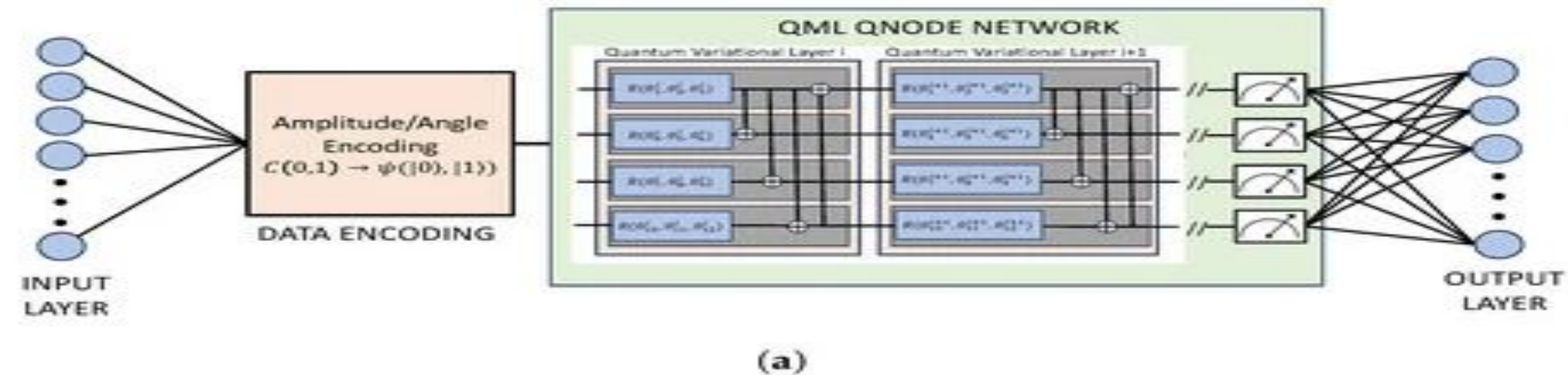
Simulations Results



(b) Velocity

FIG. 11. Comparison of the best results of (a) stream function and (b) velocity achieved by PI-QNN-(I, II) and PINN, and their absolute deviations from the exact solution.

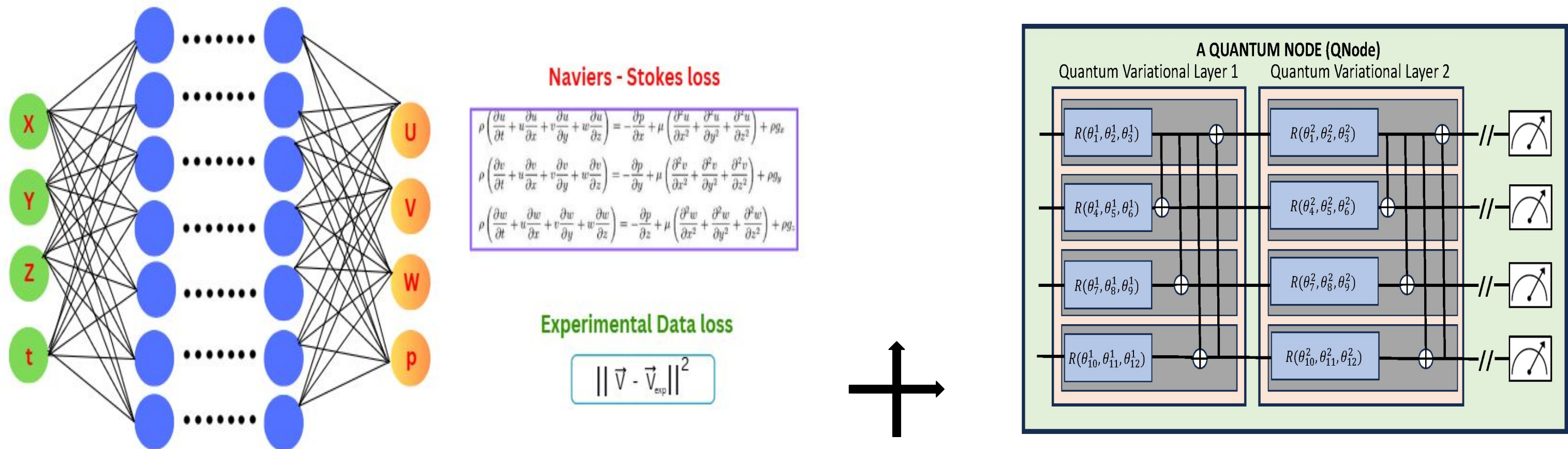
Quantum Physics Informed Neural Networks



Quantum node layer combined with classical layers

Aim

Combining Quantum with Physics-Informed Neural Networks Neural Operators, Geometry Aware Operator Transformers



**Bringing Quantum Advantage through AI
(Hybrid Approaches on QPU/GPUs Hardware)**

Variants of PINNs

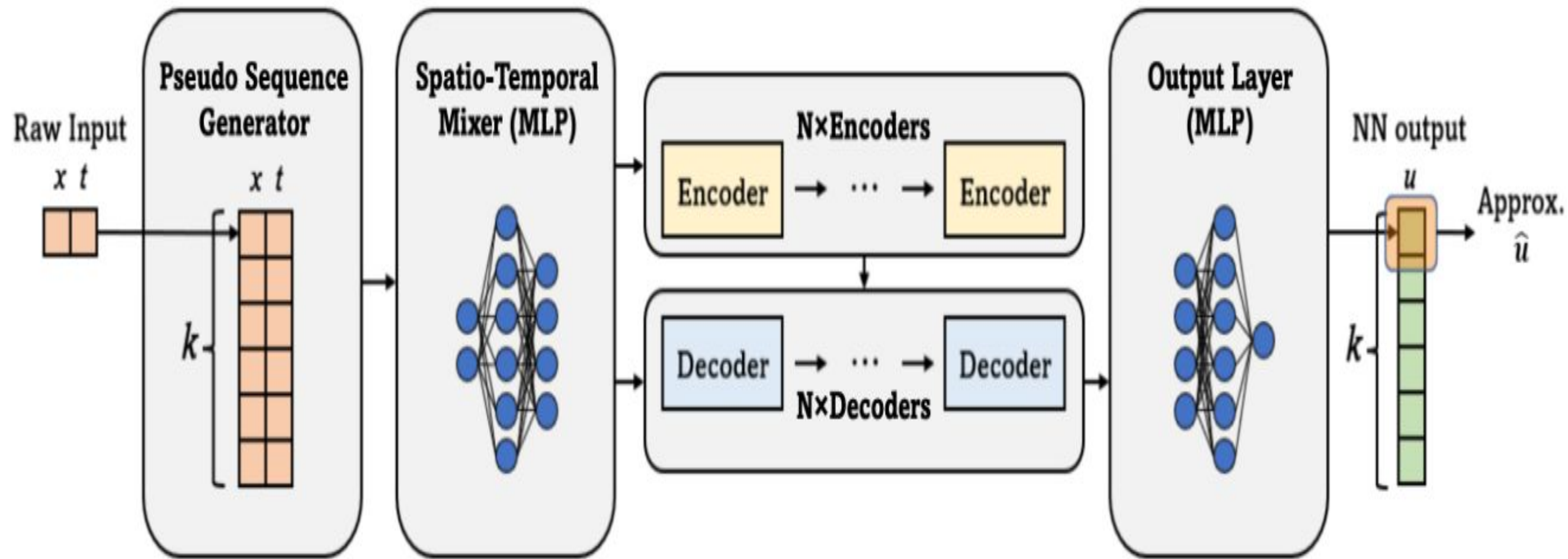


Figure 1: Architecture of proposed PINNsFormer. PINNsFormer generates a pseudo sequence based on pointwise input features. It outputs the corresponding sequential approximated solution. The first approximation of the sequence is the desired solution $\hat{u}(x, t)$.

UPS

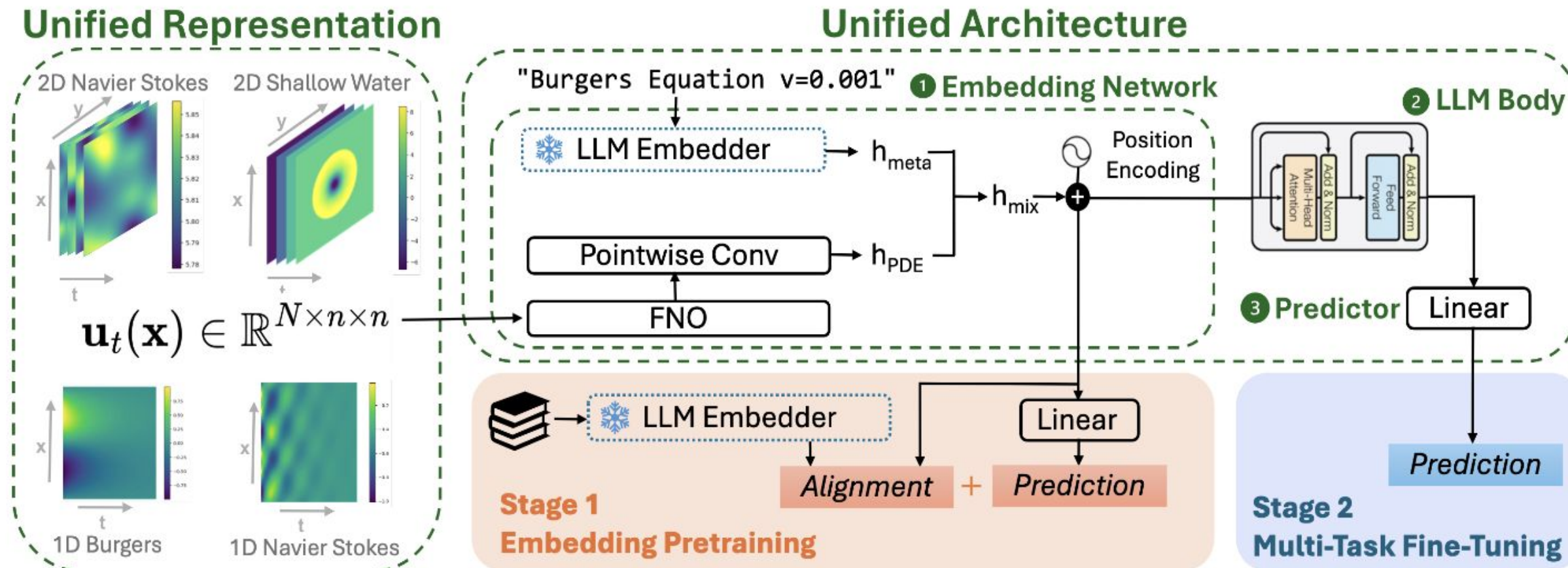


Figure 1: To adapt pretrained LLMs for PDE solving, UPS first transforms PDE of different dimensions, channels, and resolutions into a *unified representation* (left panel). Then, the data is processed with a *unified architecture* that integrates FNO layers, PDE metadata, and LLMs (right panel). The architecture is trained in two stages. In stage 1, we pretrain the embedding network using a joint loss that simultaneously optimizes (i) the distribution similarity between PDE features and text embeddings to align the modalities, and (ii) the prediction performance of extracted PDE features. In stage 2, we fine-tune the entire model on a dataset that combines multiple families of spatiotemporal PDEs with varying domain dimensions, initial/boundary conditions, and coefficients. UPS achieves competitive results with significantly better sample-efficiency than existing methods.

Variants of PINNs

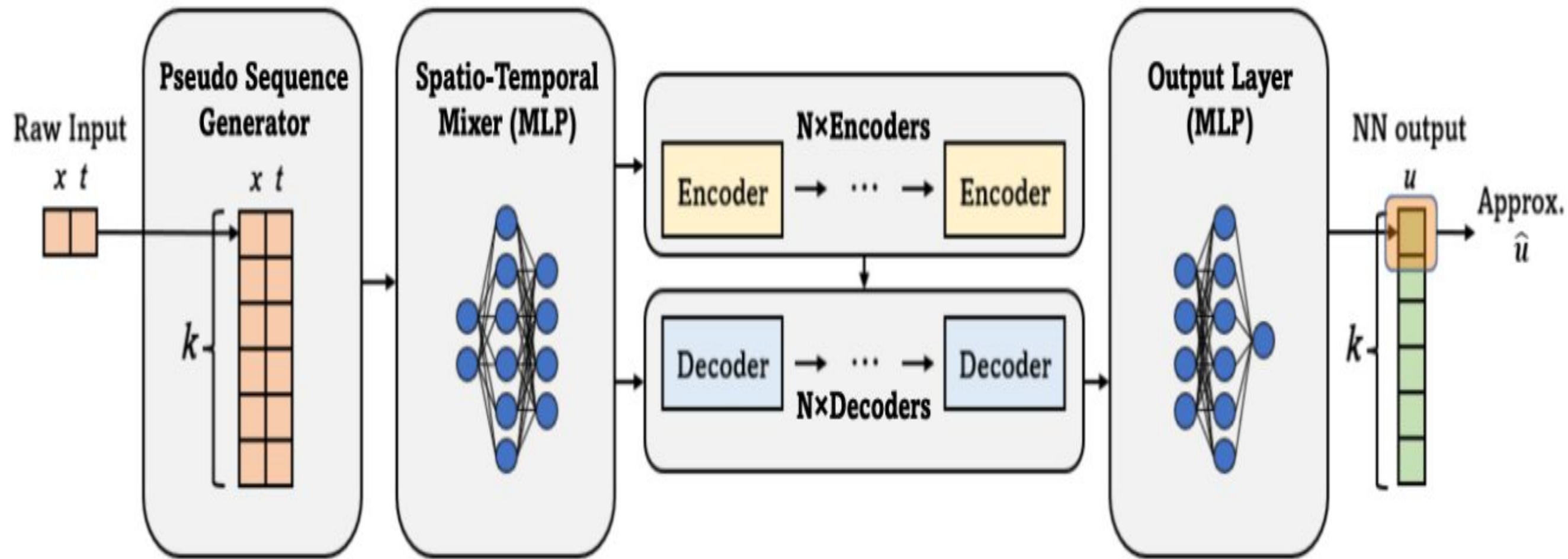
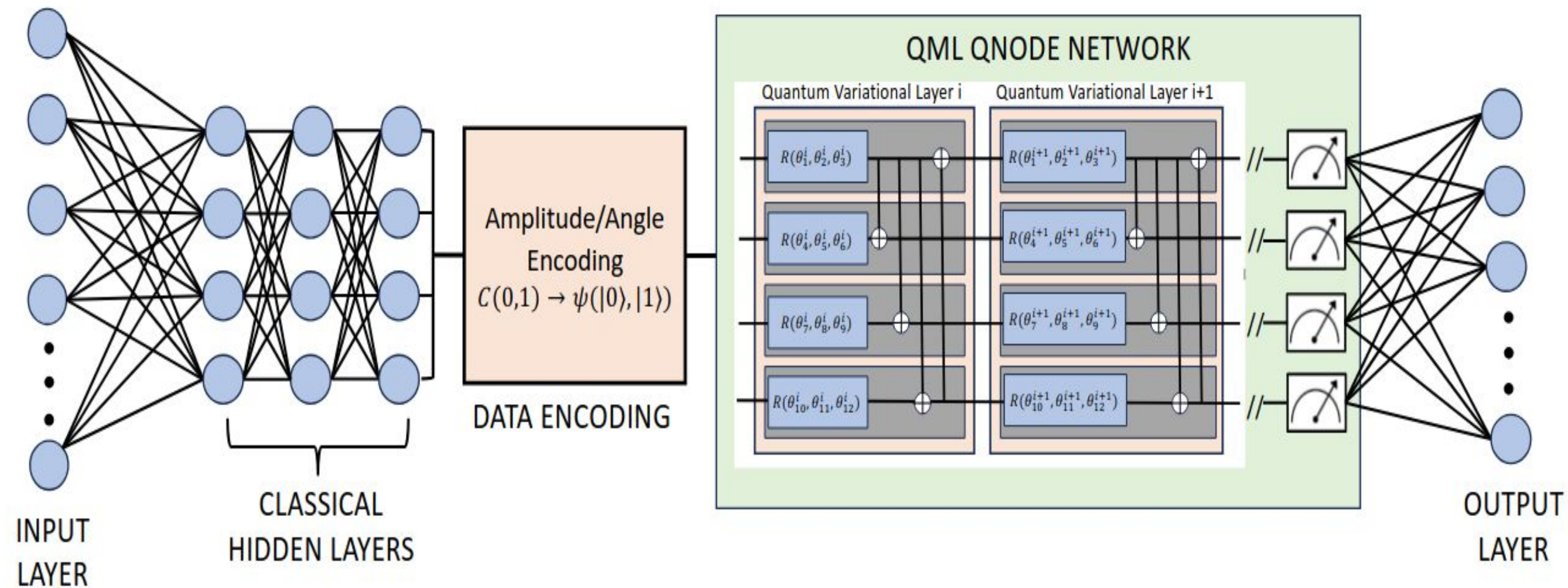


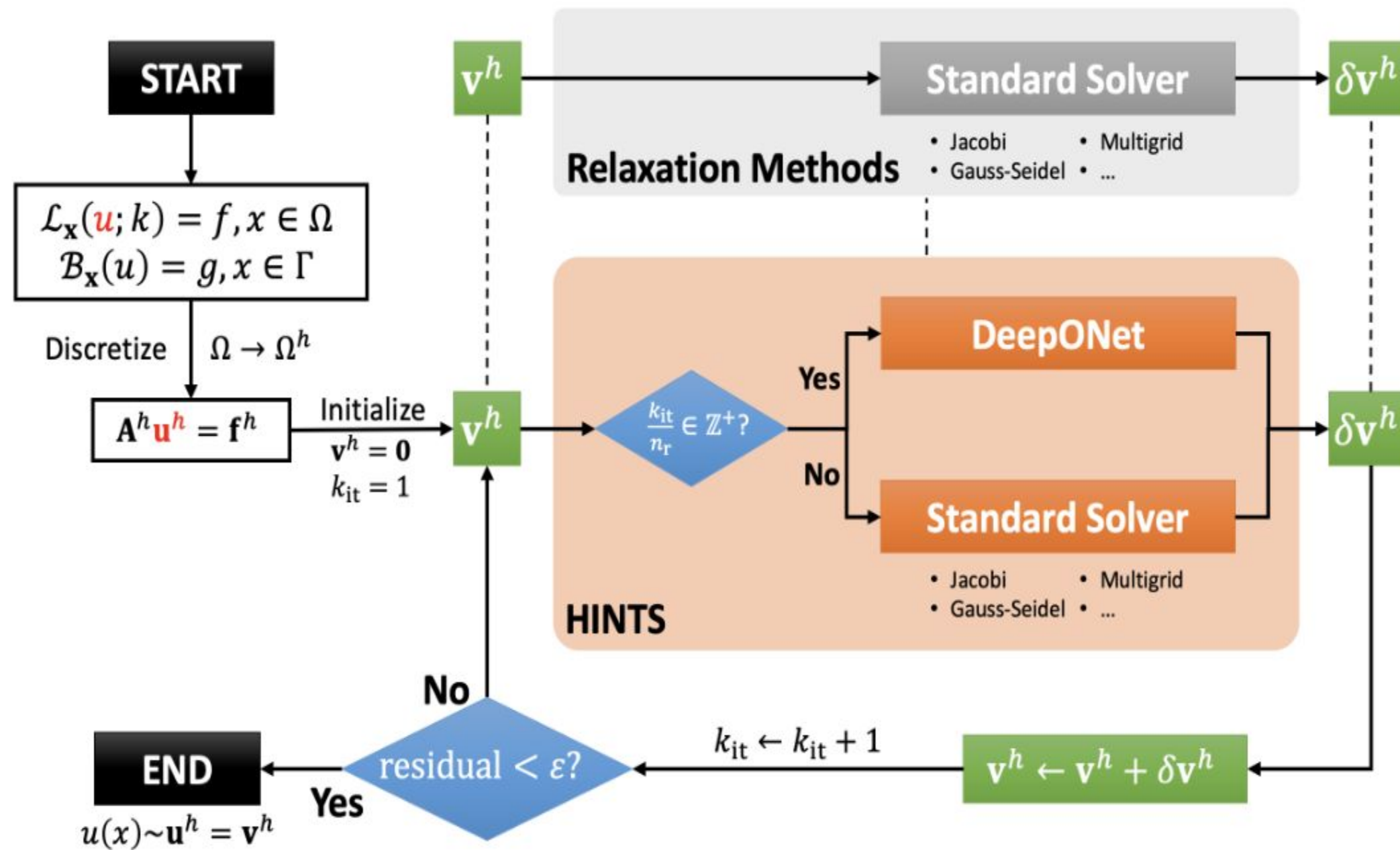
Figure 1: Architecture of proposed PINNsFormer. PINNsFormer generates a pseudo sequence based on pointwise input features. It outputs the corresponding sequential approximated solution. The first approximation of the sequence is the desired solution $\hat{u}(x, t)$.

Quantum Physics Informed Neural Networks



Quantum node layer combined with classical layers

HINTS



Advantages of PINNs

1. **Multi-physics Integration:** Fluid Dynamics, heat transfer, and chemical kinetics
2. **Data Efficiency:** By incorporating physical laws, accurate predictions even with limited experimental data
3. **Interpretability:** physics-informed nature of PINNs allows for better interpretability of results
4. **Adaptability:** different processes and configurations by adjusting equations and boundary conditions

Simulation Results

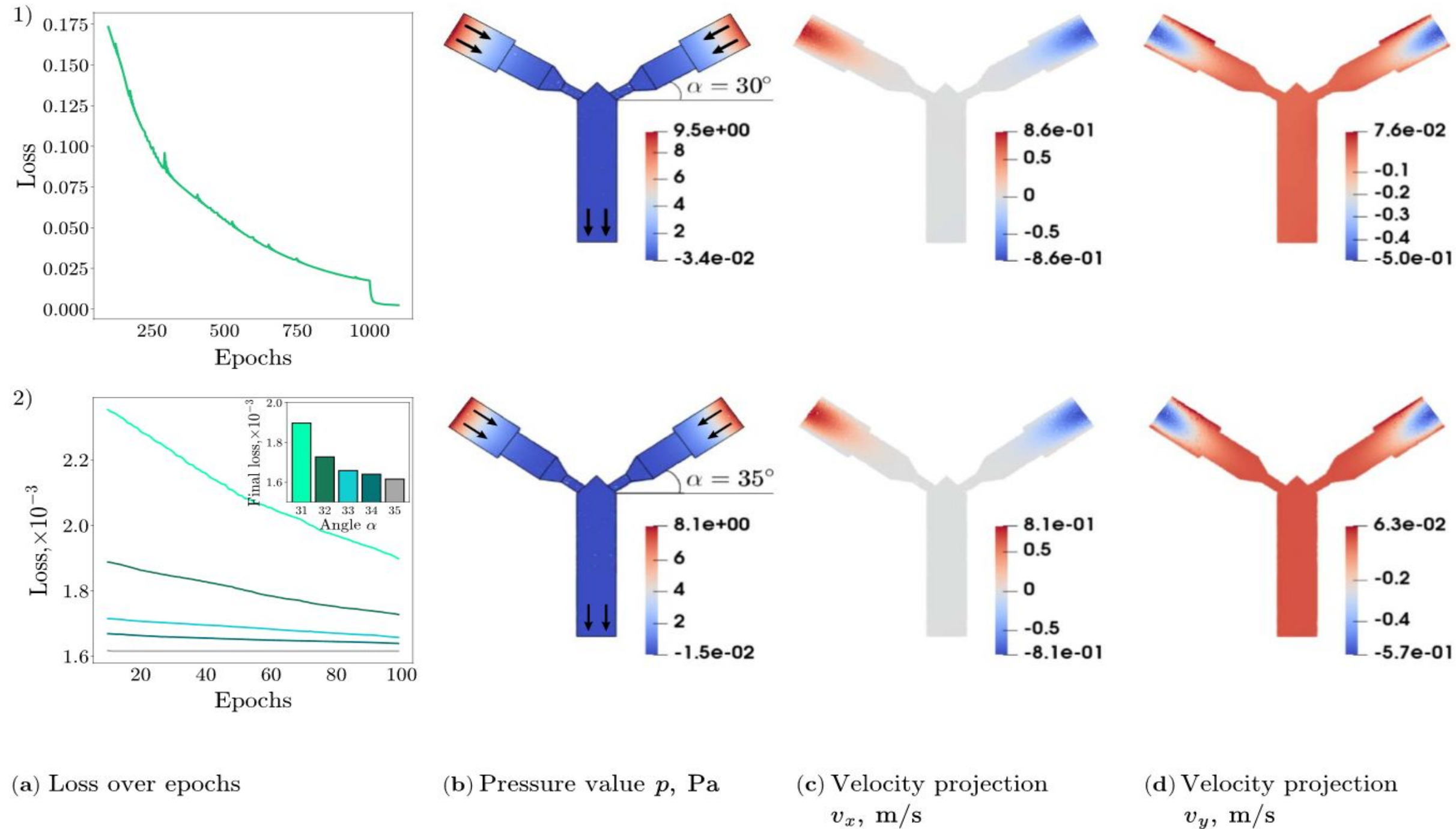


Figure 4. (1) Classical model. (a) Loss curve. The classical PINN managed to learn a solution near the entry point well. However, it vanishes to zero further down the mixer. (2) Transfer learned models. (a) Loss curve. The transfer-learned models trained better with each iteration, surpassing the original model. (b)–(d) Distributions of the fluid pressure and velocities for the last model with $\alpha = 35^\circ$.

Simulations Results

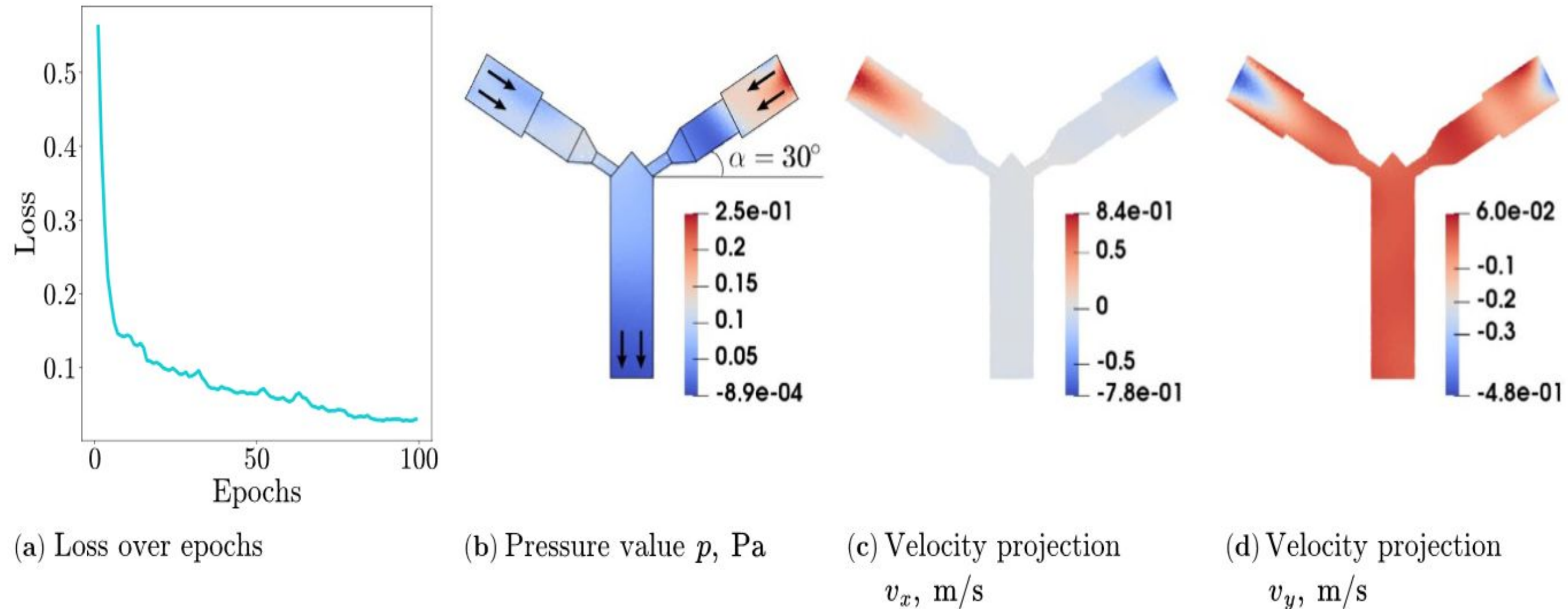
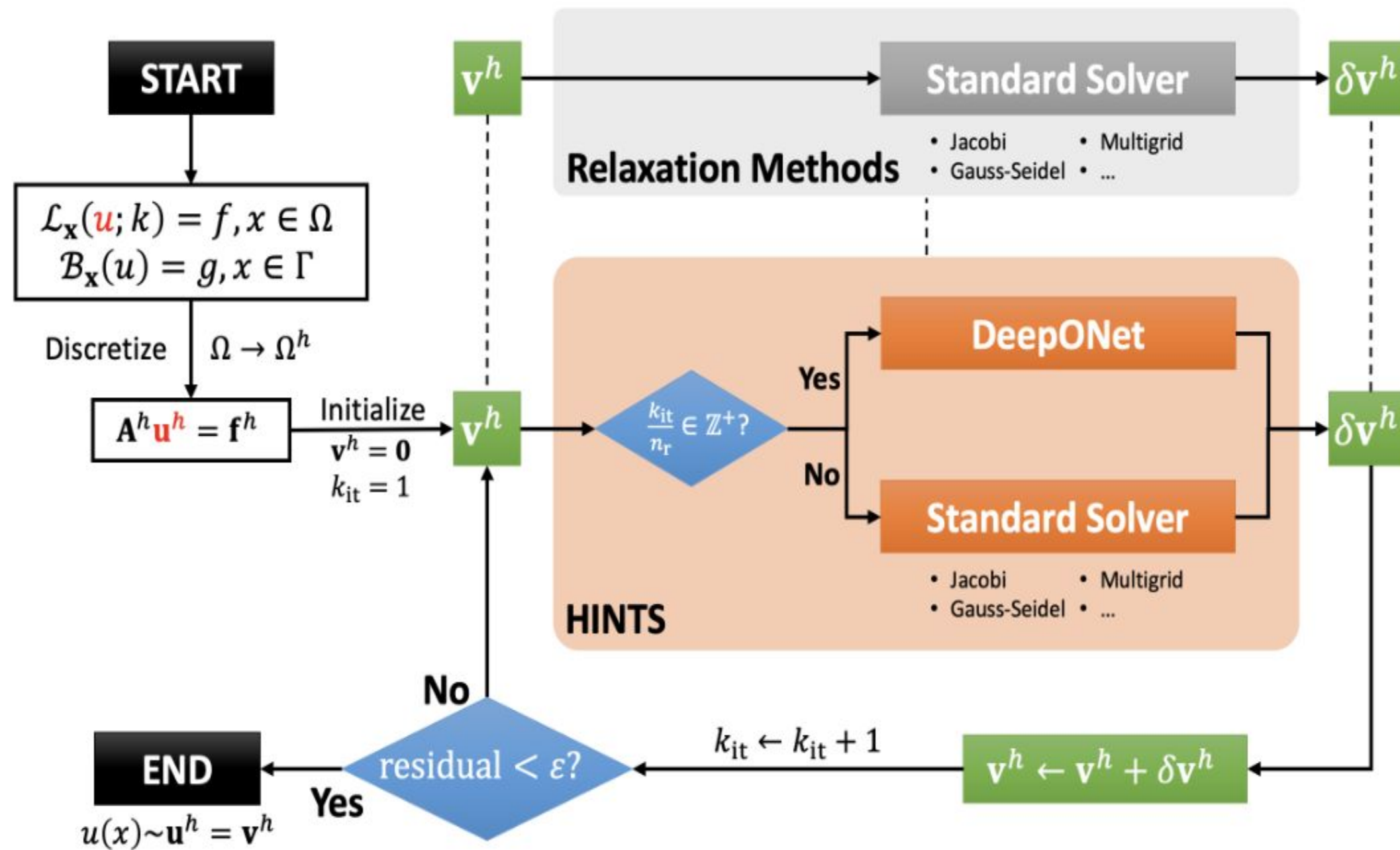
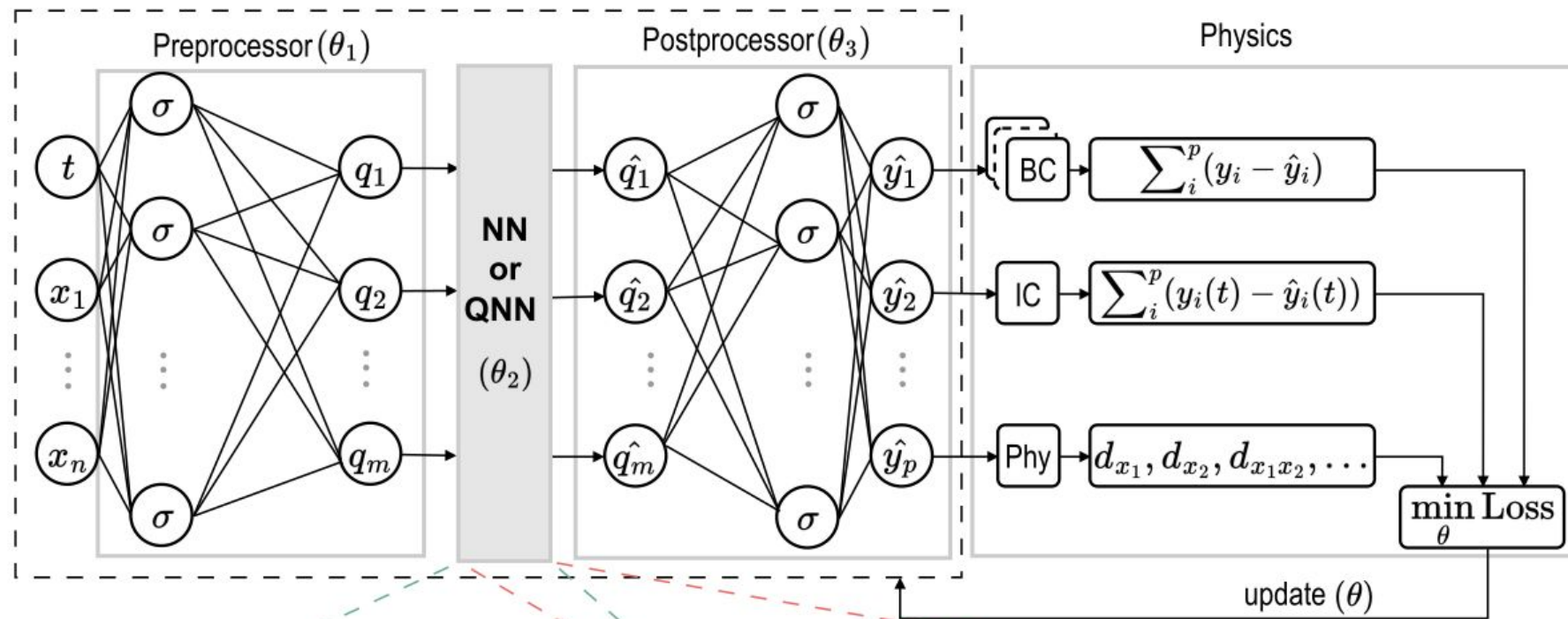


Figure 5. Hybrid quantum model. (a) Loss curve. (b)–(d) Distribution of the fluid pressure and velocities for the HQPINN at ($\alpha = 30^\circ$). Similarly to the classical model, a non-trivial solution near the entry point is present. However, there is an asymmetry between the left and the right pipe, which should not be there. This could be an effect of the data encoding strategy.

HINTS



A practical case study. Higher generalization.



Continuous variable (CV) quantum neural network can emulate a classical non-linear affine transformation – a neuron.

$$L|x\rangle = |\Phi(Mx + b)\rangle = \Phi(D \circ U_2 \circ S \circ U_1)$$

This is not always granted. Topic under study also inside QuaSi.

Key advantage : 80-90% reduction of the training volume data, same solution quality.

